A

PROJECT REPORT

on

Food Delivery Wala

Submitted in the Partial Fulfillment of the Requirement for the Award of

Bachelor of Technology

in

Computer Science And Engineering (CSE)

Submitted By

Bharat Kumar(2204850100068)

Vishvjeet Gupta(2204850100198)

Ashish Kumar(2204850100056)

Yadvesh Yadav(2204850100200)

Under the Guidance of

Shivam Kumar Srivastava

(Assistant Professor)

Department of Computer Science And Engineering

SRIMT, Lucknow, UP

SR INSTITUTE OF MANAGEMENT & TECHNOLOGY, LUCKNOW, UP

(Recognized by AICTE, Govt. of India & Affiliated to Dr. A.P.J. Abdul Kalam Technical University, Lucknow, UP)

AKTU College Code – 485

(2025-26)

SR INSTITUTE OF MANAGEMENT & TECHNOLOGY
Approved by AICTE, New Delhi & Affiliated to Dr. APJ Abdul Kalam Technical University, UP, Lucknow
NH-24, Sitapur Road, Bakshi Ka Talab, Lucknow – 226201, UP
Website: www.srimt.co.in , Email : hodcs@srgilucknow.in , Mobile No.: +91 9793000005,6,7 / 9793000045

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Vision – Institute

To evolve into an Institution imparting high quality technical education infused with core essentials of "**Shiksha",** "**Sanskar"** and "**Rojgar".**

Mission – Institute

M1: To strive for quality education of students by facilitating technology enabled holistic learning programs.
M2: To inculcate principles of professional behavior that requires adherence to the values of team work social conduct for the welfare of society.
M3: To provide necessary skills needed for entry into a global workforce with job assurance as well as facilitate training and support that help in developing the entrepreneurial abilities.

Department – Vision

The Computer Science & Engineering Department aims to explore new avenues and dimensions for making human life easier by offering students with the essential computational skills to develop technology that serves the society by enhancing employability, deep understanding of ethical values and responsibilities.

Department – Mission

M1: To ensure student's progress through regular practice and implementation of new ideas with an innovative approach
M2: To promote an environment of high moral values and discipline where students can learn and Contribute towards the growth of self and society.
M2: To further an environment of entrepreneurship and professionalism of high excellence and calibre, Giving professionals opportunities for employment and growth

SR INSTITUTE OF MANAGEMENT & TECHNOLOGY
Approved by AICTE, New Delhi & Affiliated to Dr. APJ Abdul Kalam Technical University, UP, Lucknow
NH-24, Sitapur Road, Bakshi Ka Talab, Lucknow – 226201, UP
Website: www.srimt.co.in , Email : hodcs@srgilucknow.in , Mobile No.: +91 9793000005,6,7 / 9793000045

Department PEOs

- To ensure that Students possess the understanding of engineering & computer science to excel in chosen career path.
- To ascertain that students demonstrate professional conduct, ethical attitude, cognizance of social & Environmental issues, leadership and lifelong learning.
- To create and provide opportunities for the sustainable development of students through practical Experience and training programs.

Department PSOs

- The ability to analyze and solve real life problems through the application of fundamental knowledge of maths, science and engineering.
- Understand the state of the art in the emerging areas of research in computer science and engineering, formulate problems from them and perform original work to contribute in the advancement of the state of the art.
- To adapt to evolving technical challenges and changing career opportunities by effectively communicating ideas and promoting collaboration with other members of engineering teams.

SR INSTITUTE OF MANAGEMENT & TECHNOLOGY
Approved by AICTE, New Delhi & Affiliated to Dr. APJ Abdul Kalam Technical University, UP, Lucknow
NH-24, Sitapur Road, Bakshi Ka Talab, Lucknow – 226201, UP
Website: www.srimt.co.in , Email : hodcs@srgilucknow.in , Mobile No.: +91 9793000005,6,7 / 9793000045

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

PROGRAM OUTCOME

PO 1 Engineering Knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of ***complex engineering problems.***

PO 2 Problem Analysis: Identify, formulate, review research literature, and analyze ***complex engineering problems*** reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO 3 Design/development of solutions: Design solutions for ***complex engineering problems*** and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO 4 Conduct investigations of *complex* problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions

PO 5 Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to ***complex engineering*** activities with an understanding of the limitations.

PO 6 The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO 7 Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO 8 Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice

PO 9 Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO 10 Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO 11 Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO 12 Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

COURSE OUTCOME

CO1: In a specialization domain of choice, students will be able to choose an appropriate topic for study and will be able to clearly formulate & state a research problem and analyze and understand the real-life problems and apply their knowledge to get programming Solution.

CO2: For a selected Project topic, students will be able to compile the relevant literature and frame hypotheses for project as applicable and engage in the creative design process through the integration and application of diverse technical knowledge and expertise to meet customer need and address socials issues.

CO3: For a selected topic, student manager will be able to plan a research design including the sampling, observational, statistical and operational designs if any and Use the various tools and techniques, coding practices for developing real life solution to the problems.

CO4: For a selected topic, student manager will be able to compile relevant data, interpret & analyze it and test the hypotheses wherever applicable and Find out the error in software solution and establishing the process to design maintainable software solution.

CO5: Based on the analysis and interpretation of the data collected, students will be able to arrive at logical conclusions and propose suitable recommendations on the research problem. Student manager will be able to create a logically coherent project report and will be able to defend work in front of a panel of examiners and Write the report about what they are doing in project and learning the team skills.

College Code: 485

SR INSTITUTE OF MANAGEMENT & TECHNOLOGY

Approved by AICTE, New Delhi & Affiliated to Dr. APJ Abdul Kalam Technical University, UP, Lucknow

NH-24, Sitapur Road, Bakshi Ka Talab, Lucknow – 226201, UP

Website: www.srimt.co.in , Email : hodcs@srgilucknow.in , Mobile No.: +91 9793000005,6,7 / 9793000045

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CERTIFICATE

This is to certify that the project entitled **"Food Delivery Wala"** is a bonafide record of the semester work done by **Bharat Kumar(2204850100068), Vishvjeet Gupta(2204850100198), Ashish Kumar(2204850100056), Yadvesh Yadav(2204850100200)** under my supervision and guidance, in partial fulfillment of the requirements for the Outcome-based Education Paradigm in Department of Computer Science & Engineering from **SR Institute of Management & Technology, Lucknow** for the academic year 2025-2026.

Shivam Kumar Srivastava
(Assistant Professor)
Dept. of Computer Science & Engineering
SRIMT, Lucknow, UP

Vinay singh
(HoD)
Dept. of Computer Science & Engineering
SRIMT, Lucknow, UP

Name & Signature
Internal Examiner
Dept. of Computer Science And Engineering
SRIMT, Lucknow, UP

Name & Signature
External Examiner
Dept. of Computer Science And Engineering

Place: Lucknow

Date: …….

DECLARATION

We declare that the project entitled, **"Food Delivery Wala"** represents our ideas in our own words and carried out by us under the guidance of **Shivam Kumar Srivastava** Department of Computer Science & Engineering, **SR Institute of Management & Technology, Lucknow, UP.** We have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission.

Bharat Kumar(2204850100068)
Vishvjeet Gupta(2204850100198)
Ashish Kumar(2204850100056)
Yadvesh Yadav(2204850100200)

Place: Lucknow

Date: ……

ACKNOLEDGEMENT

I would like to express my deep gratitude and sincere thanks to all the people who have helped and guided me throughout the completion of this final year project. First and foremost, I would like to thank my project guide, **Shivam Kumar Srivastava**, Assistant Professor, Department of Computer Science & Engineering, SR Institute of Manage-ment and Technology, Lucknow, for their constant support, valuable guidance, and encouragement throughout the development of this project. Their knowledge and expertise have been invaluable in shaping this work. I am also deeply thankful to the Head of the Department, for providing all the necessary facilities and resources required for the completion of this project. I extend my sincere appreciation to all the faculty members of the department for their continuous motivation and academic support throughout my studies. I also thank my friends and classmates who provided suggestions, feedback, and encouragement during the project development phase. Last but not least, I am truly grateful to my parents and family for their unconditional love, moral support, and encouragement, without which this project would not have been possible.!

Bharat Kumar(2204850100068) _____

Vishvjeet Gupta(2204850100198) _____

Ashish Kumar(2204850100056) _____

Yadvesh Yadav(2204850100200) _____

ABSTRACT

In the current era of rapid digital transformation, e-commerce has emerged as one of the most significant and widely adopted domains in the world of technology. With millions of people across the globe relying on online platforms for purchasing products and services, the demand for efficient, reliable, and user-friendly e-commerce systems has grown tremendously. The traditional method of physically visiting stores, comparing products, and completing purchases has gradually given way to digital alternatives that offer convenience, variety, and speed. **Food Delivery Wala** is a full-featured, web-based E-Commerce Application developed using the MERN Stack, which comprises MongoDB, Express.js, React.js, and Node.js. The platform is designed to provide users with a seamless and enjoyable online shopping experience, offering a comprehensive set of features including product browsing, product search and filtering, cart management, secure order placement, and online payment processing. The system is divided into two primary modules: the Customer Module and the Admin Module. The customer-side allows registered users to browse products by category, add items to the shopping cart, manage their Wishlist, place orders, and track their order history. The admin panel provides the system administrator with complete control over product management, order management, user management, and system analytics. The MERN stack was selected for this project because it enables full-stack JavaScript development using a single programming language across both the frontend and backend. React.js provides a highly interactive and responsive user interface, Node.js with Express.js handles server-side logic and RESTful API development efficiently, and MongoDB offers a flexible, schema-less NoSQL database for data storage and management.

CONTENT

Chapter 1: Introduction

Background, Objective, Purpose & Scope, Limitation

1. Introduction

BTech Food Delivery Wala" is a full-stack web-based application developed using the MERN stack, which includes MongoDB, Express.js, React.js, and Node.js. The system is designed to provide users with a seamless and efficient platform for ordering food online. It allows customers to create accounts, log in securely, browse food items categorized into different sections, add items to a shopping cart, and place orders with integrated online payment support.

The application also includes a dedicated admin panel that enables administrators to manage food items, update menu categories, monitor incoming orders, and control order status in real time. The system ensures smooth communication between the frontend and backend using RESTful APIs, enabling dynamic data exchange and responsiveness.

This project was developed as part of the final year B.Tech curriculum under the Computer Science and Engineering department at SR Institute of Management & Technology. The aim was to implement real-world concepts of full-stack development and demonstrate the integration of modern web technologies into a scalable and practical solution.

The application emphasizes usability, security, and performance. It utilizes JSON Web Tokens (JWT) for authentication, bcrypt for password encryption, and Stripe for secure online payment processing. Overall, the system reflects a real-world implementation of an e-commerce-style food delivery platform.

1.1 Background

In recent years, the rapid advancement of internet technologies and the widespread use of smartphones have significantly changed consumer behavior. People increasingly prefer online platforms for everyday services, including food ordering. Traditional methods such as ordering via phone calls or visiting restaurants involve several limitations, including human errors, lack of transparency, and inefficiency in order management.

The emergence of digital food delivery platforms has revolutionized the industry by providing users with convenience, real-time updates, and a wide variety of food choices. Popular platforms have set benchmarks in terms of user experience, fast delivery, and secure payment systems. These systems leverage modern web technologies to handle large-scale operations efficiently.

Inspired by such advancements, this project aims to replicate and simplify the core functionalities of a food delivery system using the MERN stack. The use of MongoDB provides flexibility in handling unstructured data, while React.js enables the creation of dynamic and responsive user interfaces. Node.js and Express.js ensure efficient backend processing and API management.

Additionally, the project incorporates modern tools and libraries such as:

- React Context API for global state management
- Axios for handling HTTP requests
- JWT (JSON Web Token) for secure and stateless authentication
- bcrypt for password hashing and security
- Multer for handling image uploads
- Stripe for secure and reliable payment processing

This combination of technologies allows the system to be scalable, maintainable, and aligned with current industry standards.

1.2 Objective

The primary objectives of this project are to design, develop, and implement a fully functional online food delivery web application using modern web development technologies. The key objectives include:

Development of a Full-Stack Application

To build a complete web application using the MERN stack that integrates frontend, backend, and database functionalities seamlessly.

User Authentication and Security

To implement a secure authentication system using JWT tokens and bcrypt hashing to protect user credentials and ensure safe access.

Interactive User Interface

To design a responsive and user-friendly frontend using React.js, enabling smooth navigation, dynamic content rendering, and real-time cart updates.

Backend API Development

To create RESTful APIs using Node.js and Express.js that handle business logic, including user management, order processing, and data operations.

Payment Gateway Integration

To integrate Stripe payment services for secure online transactions, ensuring reliability and user trust during payment processing.

Admin Panel Functionality

To develop an administrative interface for managing food items, categories, and customer orders efficiently.

Database Management

To use MongoDB for storing application data such as user details, food items, and orders, ensuring flexibility and scalability.

Real-Time Data Handling

To enable real-time synchronization between frontend and backend, ensuring that users receive up-to-date information regarding their orders.

1.3 Purpose and Scope

Purpose

The primary purpose of this project is to demonstrate the practical implementation of full-stack web development concepts by building a real-world application. The system addresses common problems associated with traditional food ordering methods, such as:

- Manual errors in order placement
- Lack of order tracking and transparency
- Dependence on cash-based transactions
- Limited accessibility to menus

By providing a digital solution, the project enhances efficiency, improves user experience, and ensures secure transactions. It also serves as a learning platform for understanding system design, API development, and database integration.

Scope

The scope of the project defines the boundaries and functionalities included in the system:

- Customer Web Portal

User registration and login

Browsing food items by categories

Adding/removing items from the cart

Placing orders and making payments

Viewing order details

- Backend System

RESTful API development

Authentication and authorization

Data processing and storage

Cart synchronization across sessions

Order management and status updates

- Admin Panel

Adding, updating, and deleting food items

Uploading food images

Viewing all customer orders

Updating order statuses

- Out of Scope

The following features are not included in the current version of the project:

Native mobile applications (Android/iOS)

Multi-restaurant support system

Real-time GPS-based delivery tracking

Push notifications

1.4 Limitation

Despite providing essential functionalities, the system has certain limitations that can be addressed in future improvements:

- Local Deployment
  The application currently runs on a localhost environment and requires deployment on cloud platforms for real-world usage.

- Admin Authentication Missing
  The admin panel does not include a secure authentication mechanism, which may pose security risks.
- Incomplete Promo Code Feature
  The promo code functionality is limited to the user interface and is not connected to backend logic.
- Single Restaurant Model
  The system supports only one restaurant, limiting scalability and business expansion.
- No GPS Tracking
  There is no real-time tracking of delivery personnel, which reduces transparency for users.
- Lack of Feedback System
  Users cannot provide reviews or ratings for food items or services.
- No Push Notifications
  The system does not notify users about order updates or offers in real time.
- Scalability Constraints
  Without optimization and cloud infrastructure, the system may face performance issues under high traffic.

Chapter 2: Feasibility Study

Economic Feasibility, Technical Feasibility, Behavioral Feasibility

2.1 Economic Feasibility

Economic feasibility examines the cost-effectiveness of the project by comparing the expected benefits with the overall development and operational costs. The goal is to ensure that the system can be developed and maintained within reasonable financial limits.

In the case of this project, the development cost is significantly minimized due to the use of open-source technologies. The core technologies used—React.js, Node.js, Express.js, and MongoDB—are freely available and do not require any licensing fees. This eliminates the need for expensive software procurement, making the project highly economical.

Additionally, cloud-based services such as MongoDB Atlas provide a free tier that supports database hosting with sufficient storage and performance for small to medium-scale applications. Similarly, the Stripe payment gateway offers a free sandbox environment for testing transactions, allowing developers to implement and validate payment functionalities without incurring financial costs during the development phase.

For deployment purposes, platforms like Vercel (for frontend hosting) and Render (for backend hosting) offer free hosting tiers. These platforms provide essential features such as continuous deployment, SSL security, and scalability options at no initial cost, which is ideal for academic projects and early-stage applications.

The primary investment in this project is the time and effort contributed by the development team. Since it is developed as part of a B.Tech curriculum, no external labor costs are involved. Maintenance costs are also minimal, especially during the initial stages.

From a long-term perspective, if the application is scaled for commercial use, additional costs such as premium hosting, domain registration, advanced database plans, and marketing may be incurred. However, for the current scope, the system is highly cost-efficient and economically feasible.

Resource	Tool/Service	Cost
Frontend	React.js + Vite	Free - Open Source
Backend	Node.js + Express.js	Free - Open Source
Database	MongoDB Atlas	Free Tier

Payment Testing	Stripe Sandbox	Free for Testing
Hosting	Vercel / Render	Free Tier

2.2 Technical Feasibility

Technical feasibility evaluates whether the available technology, tools, and resources are sufficient to successfully develop and run the proposed system. It also considers the technical skills of the development team and system compatibility.

The “BTech Food Delivery Wala” application is built using the MERN stack, which is a widely adopted and well-established technology stack in modern web development. Each component of the MERN stack is mature, well-documented, and supported by a large developer community, making it reliable and easy to implement.

React.js is used for building dynamic and responsive user interfaces. Its component-based architecture allows efficient code reuse and faster development.

Node.js provides a runtime environment for executing JavaScript on the server side, enabling scalable and high-performance backend operations.

Express.js simplifies backend development by providing robust tools for creating APIs and handling HTTP requests.

MongoDB is a NoSQL database that offers flexibility in data storage and is suitable for handling dynamic and unstructured data.

The development team has gained foundational knowledge of these technologies during their B.Tech coursework, which ensures that the required technical skills are already available. This reduces the learning curve and enhances development efficiency.

Hardware Requirements

The system does not require high-end hardware. The minimum hardware configuration includes:

- Intel Core i5 processor or equivalent
- 8 GB RAM
- 256 GB SSD storage
- Stable internet connection

This configuration is sufficient for both development and testing purposes.

Software Requirements

The development and execution of the project require the following software tools:

- Node.js (version 18 or higher)
- npm (Node Package Manager) version 9 or higher
- MongoDB Atlas (cloud database service)
- Visual Studio Code (code editor)
- Git (version control system)
- Google Chrome or any modern web browser

The integration of these tools ensures smooth development, debugging, and deployment processes.

Furthermore, the application architecture is designed using RESTful APIs, which makes it modular and scalable. Security measures such as JWT authentication and password hashing using bcrypt enhance system reliability.

Overall, the project is technically feasible as it uses proven technologies, requires moderate system resources, and aligns well with the technical expertise of the development team.

2.3 Behavioral Feasibility

Behavioral feasibility focuses on the acceptance of the system by its intended users. It evaluates how easily users can adapt to the system and whether it meets their expectations and requirements.

The user interface of the “BTech Food Delivery Wala” application is designed to be simple, intuitive, and user-friendly. The layout follows common web application design patterns, ensuring that users can navigate the system without confusion. Features such as category-based browsing, cart management, and straightforward checkout processes enhance usability.

Customers who are familiar with basic internet usage and web browsing can easily interact with the application. The system does not require any specialized training, making it accessible to a wide range of users, including those with minimal technical knowledge.

The admin panel is also designed with simplicity in mind. Restaurant staff with basic computer skills can manage food items, update menus, and track orders efficiently. The interface minimizes complexity and focuses on essential functionalities to ensure ease of operation.

From a user perspective, the application addresses real-world needs such as:

- Convenient food ordering from home
- Access to a variety of menu options
- Secure online payment methods
- Improved transparency in order processing

These factors contribute to a positive user experience and increase the likelihood of adoption.

Moreover, the growing trend of online food ordering indicates a high acceptance rate for such systems. Users are already accustomed to similar platforms, which further reduces resistance to adopting this application.

In conclusion, the system demonstrates strong behavioral feasibility as it is user-centric, easy to use, and aligned with current user expectations and market trends**.**

Chapter 3: Requirement & Analysis

Problem Definition, Planning & Scheduling, Software/Hardware Requirements, SDLC Model

3.1 Problem Definition

Traditional food ordering through phone calls or in-person visits suffers from manual order errors, no real-time tracking, cash-only payments, limited menu visibility, and no digital order history. The proposed Btech Food Delivery Wala system addresses all these through: digital menu browsing, JWT-secured accounts, Stripe payments, real-time cart management, order status tracking, and admin order management.

3.2 Planning & Scheduling

3.2.1 System Flow Chart

The following flowchart illustrates the complete user journey from visiting the website to order confirmation after payment:

This flowchart represents the working of a food delivery system. It starts when a user visits the website and checks login status. If not logged in, the user must register or log in with valid credentials. After authentication, the user browses the menu, selects items, and adds them to the cart. If the cart is not empty, the user proceeds to checkout, enters the delivery address, and completes payment via Stripe. If payment fails, the process repeats; if successful, the order is confirmed. This ensures a secure, efficient, and user-friendly ordering process.

Key Points:

- User authentication (login/register)
- Menu browsing and filtering
- Add to cart functionality
- Cart validation before checkout
- Secure payment integration (Stripe)
- Order confirmation and update system

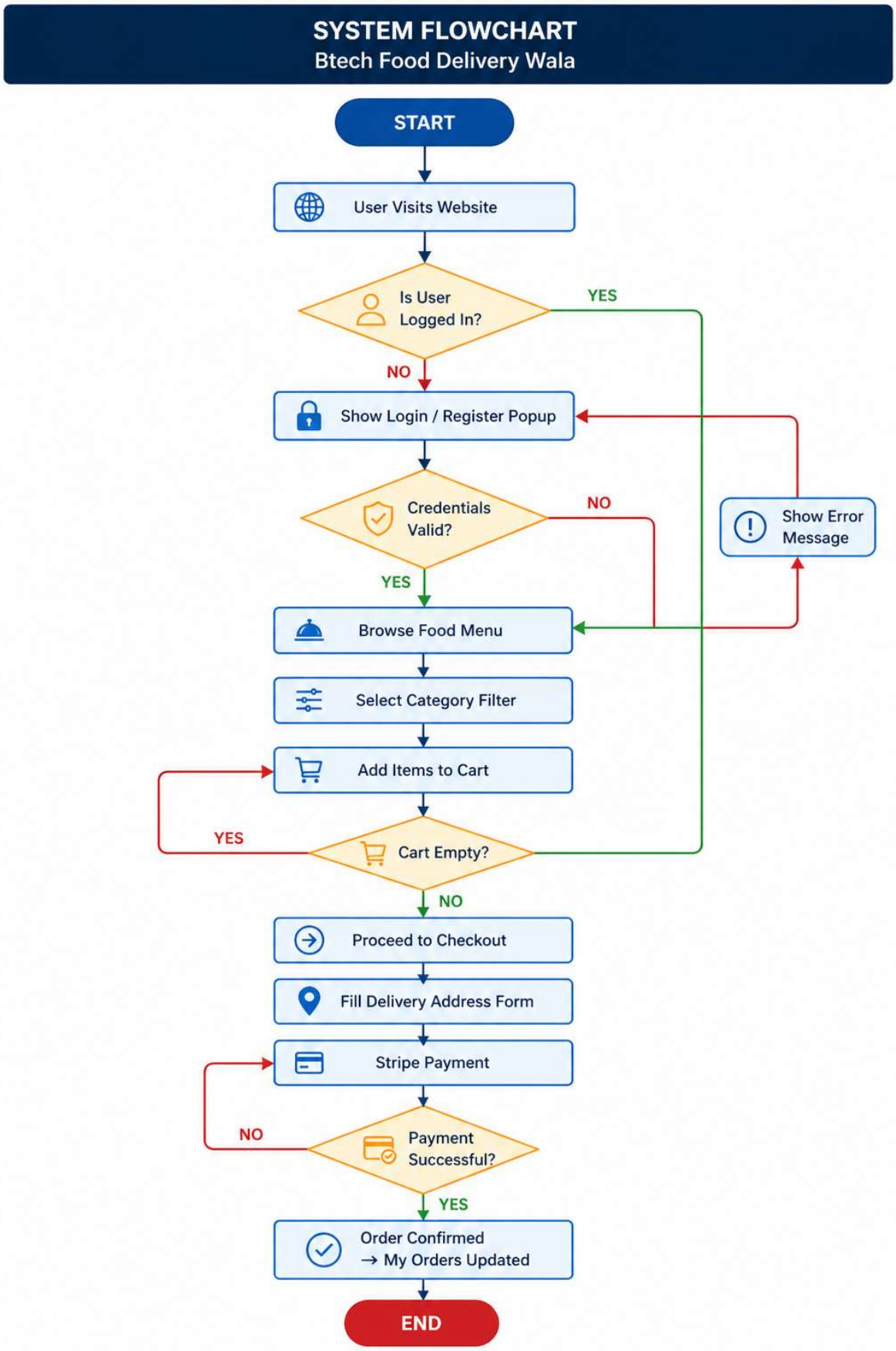


Fig 3.1: System Flow Chart – Complete User Journey

3.2.2 Gantt Chart – Project Schedule

The Gantt chart below shows the 8-week project schedule covering all phases from requirement analysis to final submission:

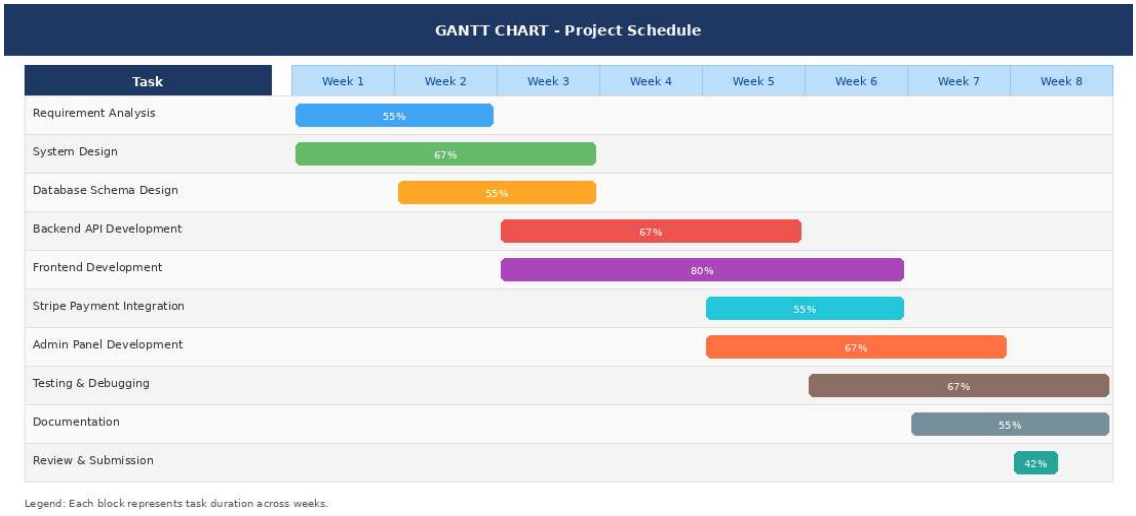
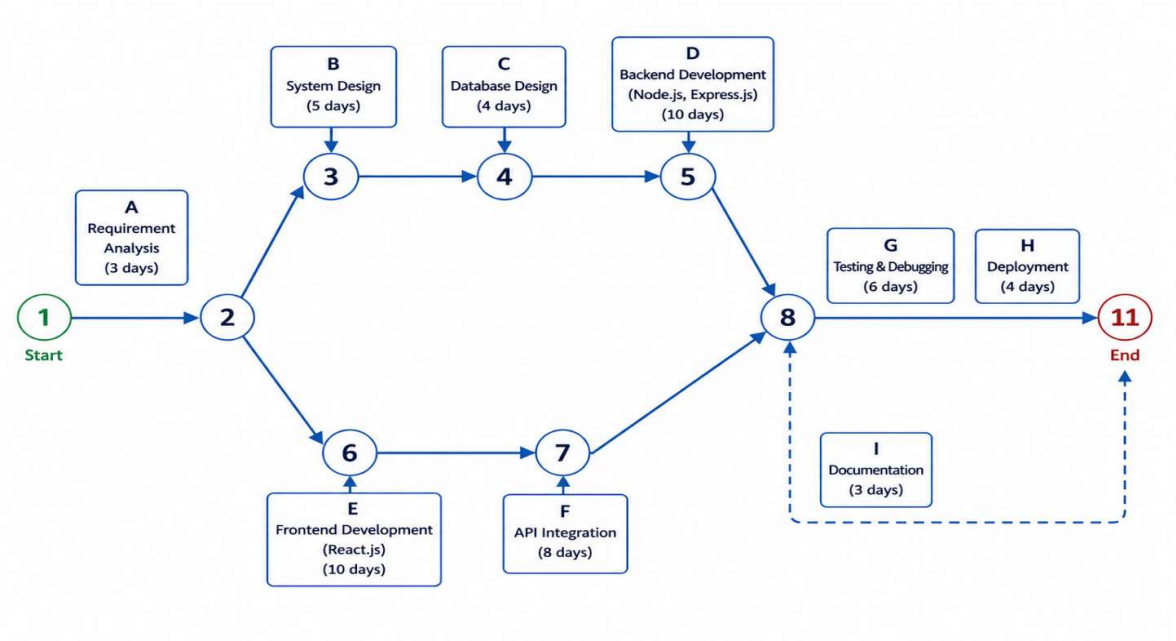


Fig 3.2: Gantt Chart – 8-Week Project Timeline

3.3 Pert Chart – Project Evaluation and Review Technique

PERT (Program Evaluation and Review Technique) is a project management tool used to analyze and represent the tasks involved in completing a project. It helps in estimating the minimum time required to complete the project and identifies the silical peth The following PERT chart illustrates the major activities involved in the development of the "Tech Food Delivery Wals" application



3.4 Software / Hardware Requirements

Hardware:

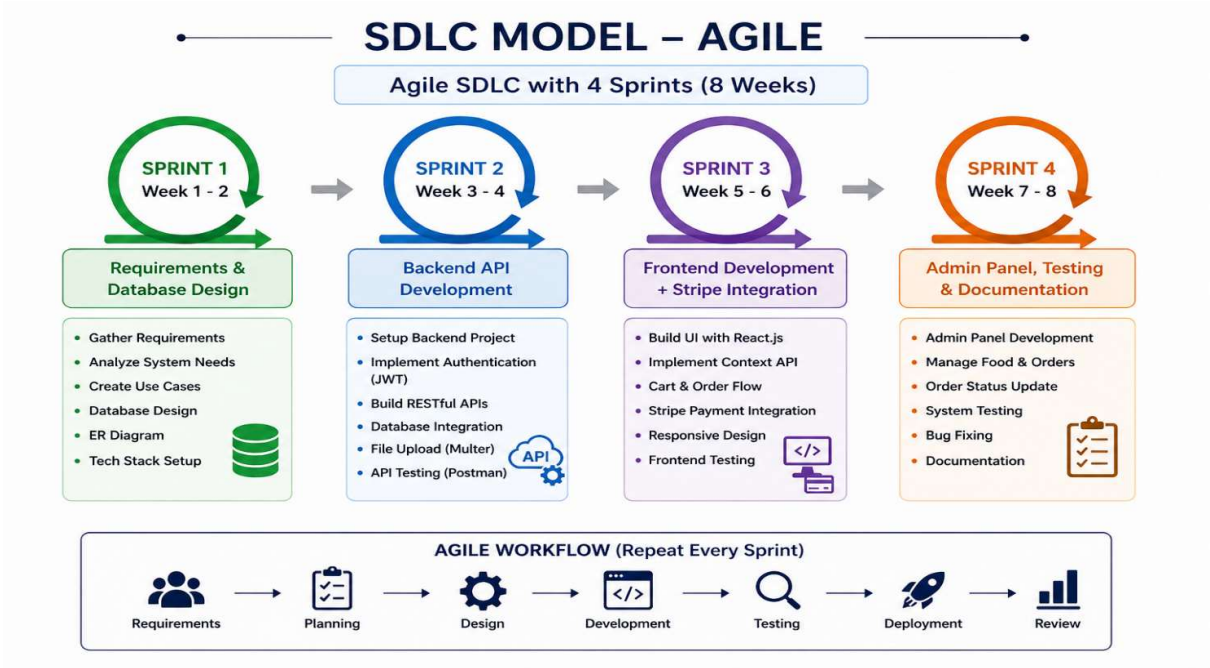
Processor: Intel Core i5+, RAM: 8GB minimum, Storage: 256GB SSD, Internet: 10 Mbps+, Display: 1366x768+.

Software:

Software	Version	Purpose
Node.js	v18.x	Backend Runtime
React.js	v18.x	Frontend Framework
MongoDB Atlas	v6.x	Cloud Database
VS Code	Latest	Code Editor
Git & GitHub	Latest	Version Control
Postman	Latest	API Testing

3.4 SDLC Model – Agile

Agile SDLC with 4 sprints: Sprint 1 (Week 1-2): Requirements & DB design. Sprint 2 (Week 3-4): Backend API. Sprint 3 (Week 5-6): Frontend + Stripe. Sprint 4 (Week 7-8): Admin panel, testing, documentation.



Chapter 4: Survey of Technology (Literature Review)

Previous Work, Technology Used, IDE, API, Tools

4. Introduction

The Survey of Technology, also known as the Literature Review, is an important part of system development that examines existing research, technologies, and tools relevant to the proposed project. It helps in understanding how similar systems have been developed, what technologies are commonly used, and what improvements can be made.

For the "BTech Food Delivery Wala" application, this chapter explores previous work in the domain of online food ordering systems and provides a detailed overview of the technologies, tools, and development environments used in the project.

4.1 Previous Work

The concept of online food ordering systems has evolved significantly over the past decade due to the rapid growth of internet connectivity and smartphone usage. Several commercial platforms have successfully implemented digital food delivery models, transforming the traditional food service industry.

Early platforms introduced the idea of browsing restaurant menus online and placing orders digitally. Over time, these systems evolved to include advanced features such as real-time order tracking, online payments, personalized recommendations, and user reviews.

In India, platforms like Zomato (established in 2008) and Swiggy (launched in 2014) played a crucial role in popularizing online food delivery services. These platforms focused on providing a user-friendly interface, a wide variety of food options, reliable delivery services, and seamless payment integration. Their success demonstrated the scalability and profitability of digital food ordering systems.

From an academic perspective, various research studies have analyzed the factors contributing to the success of such platforms. Key findings from these studies include:

- Ease of Use: Simple and intuitive user interfaces increase user engagement.
- Variety of Options: Availability of diverse food categories enhances user satisfaction.
- Reliable Delivery: Timely and accurate delivery builds trust among users.

- Secure Payment Systems: Integration of digital payment methods improves convenience and reduces dependency on cash.

Research in web development has also highlighted the effectiveness of modern full-stack technologies in building scalable applications. The MERN stack (MongoDB, Express.js, React.js, Node.js) has gained significant popularity due to its ability to use JavaScript across both frontend and backend, simplifying development and improving performance.

This project builds upon these insights by implementing a simplified yet functional food delivery system using the MERN stack. It focuses on core features such as authentication, menu browsing, cart management, and online payments while maintaining scalability and performance.

4.2 Technology Used

The "BTech Food Delivery Wala" application is developed using a combination of modern web technologies that ensure efficiency, scalability, and maintainability. These technologies are categorized into frontend, backend, and database components.

4.2.1 Frontend Technologies

The frontend of the application is responsible for user interaction and interface design. It is developed using React.js along with supporting libraries and tools.

- React.js (Version 18):
  React.js is a JavaScript library used for building dynamic and interactive user interfaces. It follows a component-based architecture, allowing developers to create reusable UI components. The use of a virtual DOM improves performance by minimizing direct manipulation of the real DOM.
- React Router DOM (Version 6):
  This library enables client-side routing in single-page applications (SPA). It allows seamless navigation between different pages without reloading the entire application, improving user experience.
- Context API:
  React Context API is used for global state management. It helps in managing shared data such as user authentication status and cart items across multiple components without the need for prop drilling.

- Axios:
  Axios is a promise-based HTTP client used for making API requests to the backend. It simplifies data fetching and ensures smooth communication between frontend and backend.
- CSS3 (Cascading Style Sheets):
  CSS3 is used for styling the application. It ensures responsive design, enabling the application to adapt to different screen sizes and devices.

4.2.2 Backend Technologies

The backend handles business logic, data processing, and communication with the database. It is developed using Node.js and Express.js along with several supporting libraries.

- Node.js (Version 18)
  Node.js is a runtime environment that allows JavaScript to be executed on the server side. It uses a non-blocking, event-driven architecture, making it efficient for handling multiple requests simultaneously.
- Express.js (Version 4)
  Express.js is a lightweight web framework for Node.js that simplifies the creation of RESTful APIs. It provides routing, middleware support, and request handling capabilities.
- Mongoose (Version 7)
  Mongoose is an Object Data Modeling (ODM) library for MongoDB. It provides a structured way to define schemas and interact with the database.
- JSON Web Token (JWT)
  JWT is used for secure user authentication. It enables stateless authentication by generating tokens that are verified on each request.
- bcrypt (Version 5)
  bcrypt is used for hashing user passwords before storing them in the database. This enhances security by preventing plain-text password storage.
- Multer
  Multer is a middleware used for handling file uploads, such as food item images, on the server.

- Stripe SDK

  Stripe is integrated to handle secure online payments. It supports transaction processing and ensures payment security.

- CORS (Cross-Origin Resource Sharing)

  CORS is used to allow controlled access to resources from different domains, enabling frontend-backend communication.

- dotenv

  dotenv is used to manage environment variables securely, such as API keys and database credentials.

4.2.3 Database Technology

- MongoDB Atlas

  MongoDB Atlas is a cloud-based NoSQL database service used to store application data. It uses a flexible BSON (Binary JSON) document model, which allows dynamic schema design.

  The database in this project consists of three main collections:

- Users Collection

  Stores user information such as name, email, password (hashed), and cart data. The cart data is embedded within the user document for efficient access.

- Foods Collection

  Contains details of food items, including name, category, price, description, and image URL.

- Orders Collection

  Stores order-related information such as user ID, ordered items, total amount, delivery address, and order status.

MongoDB's scalability and flexibility make it suitable for handling dynamic data and growing application requirements.

4.3 IDE and Development Environment

The development of the project is carried out using modern development tools and environments that enhance productivity and code quality.

- Visual Studio Code (VS Code)
  A lightweight and powerful code editor used for writing and managing code. It provides features such as syntax highlighting, extensions, debugging tools, and version control integration.
- Git
  A version control system used to track changes in the codebase and manage collaboration.
- GitHub (optional)
  Used for hosting the project repository and maintaining version history.
- Web Browser (Google Chrome)
  Used for testing and debugging the application. Chrome Developer Tools help in inspecting elements and monitoring network requests.

4.4 APIs and External Services

The application uses APIs to enable communication between different components and integrate external services.

- RESTful APIs
  The backend exposes REST APIs for handling user authentication, food management, cart operations, and order processing.
- Stripe Payment API
  Used for handling online payments securely. It manages transaction processing and ensures data security.

4.5 Tools and Utilities

Several tools are used during development to improve efficiency and debugging:

- Postman
  Used for testing API endpoints and verifying request-response cycles.
- NPM (Node Package Manager)
  Used for managing project dependencies and installing required libraries.
- MongoDB Atlas Dashboard
  Provides a graphical interface to manage database collections and monitor performance.

4.6 IDE, API, Various Tools

Tool	Type	Usage
VS Code	IDE	Primary editor with ESLint, Prettier
Vite	Build Tool	Fast React dev server & production bundler
Postman	API Testing	All REST endpoint testing during development
Git & GitHub	VCS	Version control and team collaboration
MongoDB Compass	DB GUI	Visual database inspection and queries
Stripe Dashboard	Payment	Test payment monitoring

Chapter 5: Preliminary Module Description

5.1 Use Case Diagram

The Use Case Diagram represents the interactions between different actors and the system. It provides a visual representation of how users (Customer and Admin) interact with the application to perform various tasks.

The primary actors involved in the system are:

Customer – End user who browses food items, places orders, and makes payments.

Admin – Manages food items and monitors customer orders.

External System (Stripe) – Handles secure payment processing.

The use case diagram captures all functional requirements of the system, including authentication, food browsing, cart management, order placement, and administrative controls.

Customer Interactions:

The customer interacts with the system to perform the following actions:

- Register and log in to the system
- Browse food items available in different categories
- Filter food items based on preferences
- Add or remove items from the cart
- View cart details before checkout
- Place orders and proceed to payment
- Make secure payments via Stripe
- Track order status and view order history

Admin Interactions:

The admin interacts with the system to manage operations:

- Add new food items to the menu
- Upload food images
- Remove or update existing food items
- View all customer orders
- Update order status in real time

System Interactions

The system coordinates between the frontend, backend, and external services:

- Authenticates users using secure tokens
- Processes orders and stores data
- Communicates with the Stripe API for payment handling

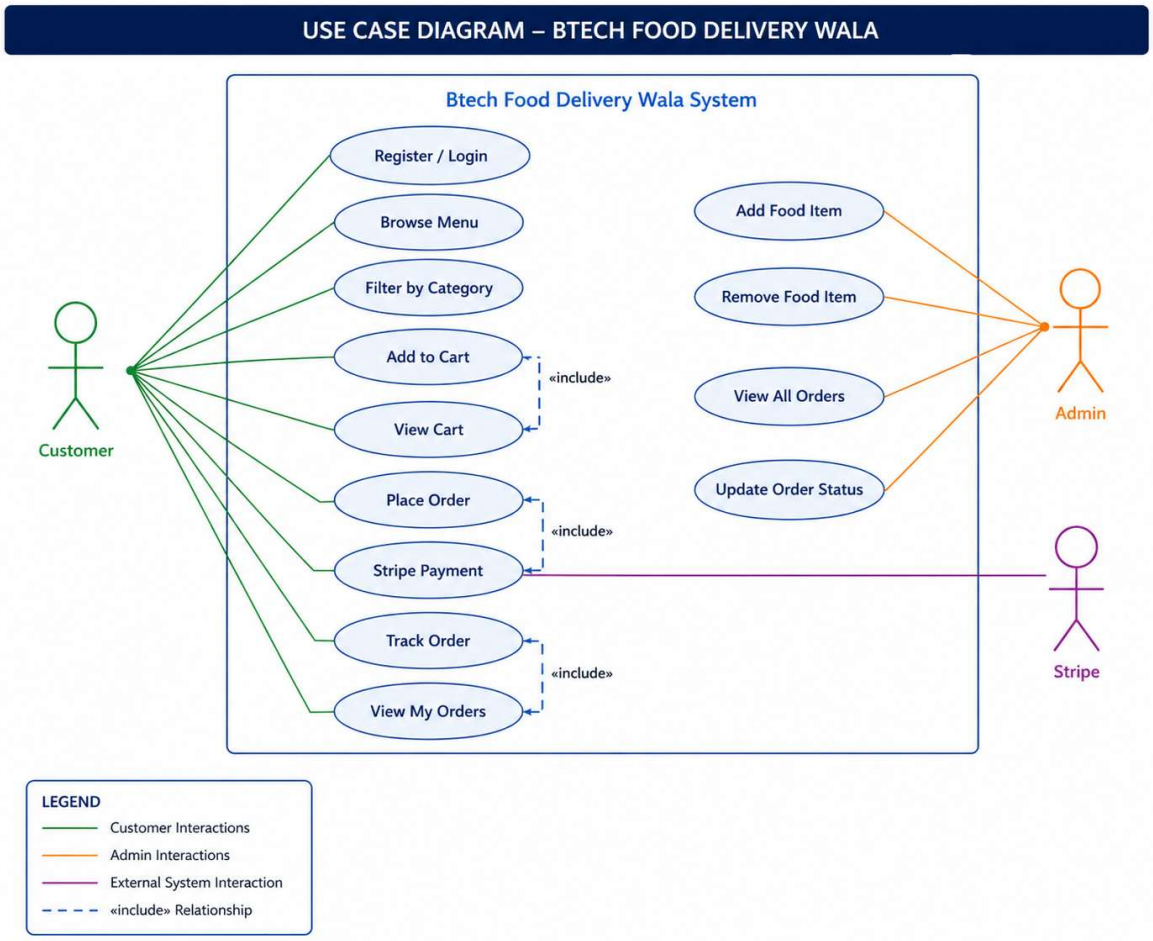


Fig 5.1: Use Case Diagram – Btech Food Delivery Wala

5.2 Module Summary

The system is divided into multiple modules based on functionality. Each module is responsible for a specific set of operations, ensuring separation of concerns and efficient system performance.

5.2.1 Customer Modules

The Customer Module focuses on user-facing features and interactions.

1. User Authentication Module

- This module handles user registration and login functionality.

- Allows new users to create accounts using email and password
- Uses JWT (JSON Web Token) for secure session management
- Encrypts passwords using bcrypt before storing in the database
- Ensures only authenticated users can access protected routes

2. Food Browsing Module

- This module enables users to explore available food items.
- Displays food items categorized into different sections
- Provides filtering options based on categories
- Shows details such as price, description, and images

Ensures a smooth and responsive browsing experience

3. Cart Management Module

- This module manages the user's shopping cart.
- Allows users to add or remove food items
- Updates item quantity dynamically
- Stores cart data persistently using database integration
- Synchronizes cart data across sessions

4. Order Placement Module

- This module handles order creation and checkout.
- Collects delivery address and order details
- Calculates total price based on selected items
- Initiates payment process using Stripe integration
- Confirms order upon successful payment

5. Order Tracking Module

- This module allows users to monitor their orders.
- Displays order history (My Orders section)
- Shows real-time order status updates
- Enables users to track progress from order placement to delivery

5.2.2 Admin Modules

The Admin Module is responsible for managing system data and operations.

1. Food Management Module

- Allows admin to add new food items
- Uses Multer middleware for uploading food images
- Stores food details in the database
- Enables removal of outdated or unavailable items

2. Order Management Module

- Displays all customer orders in a centralized dashboard
- Provides detailed information about each order
- Helps in monitoring order activity

3. Order Status Update Module

- Allows admin to update order status
- Status options include:
    a) Food Processing
    b) Out for Delivery
    c) Delivered
- Updates are reflected in the customer's order tracking interface

---

5.2.3 System Modules

The System Module handles backend processes and integrations.

1. Authentication Middleware

- Verifies JWT tokens for protected routes
- Ensures secure communication between client and server

2. Password Security Module

- Uses bcrypt hashing algorithm
- Protects user credentials from unauthorized access

3. Database Management Module

- Performs CRUD (Create, Read, Update, Delete) operations
- Manages collections for users, food items, and orders
- Ensures data consistency and integrity

4. Payment Processing Module

- Integrates Stripe payment gateway
- Creates secure payment sessions
- Handles transaction responses

5. Payment Verification Module

- Confirms whether payment is successful
- Updates order status accordingly
- Prevents fraudulent or incomplete transactions

6. Cart Persistence Module

- Stores cart data in the database
- Maintains cart state even after user logout
- Ensures a seamless shopping experience

Chapter 6: System Design

DFD Level 0, Level 1, Level 2, ER Diagram, Data Structure

6.1 Data Flow Diagrams (DFD)

6.1 Data Flow Diagrams (DFD)

Data Flow Diagrams (DFDs) are graphical representations of how data flows within a system. They illustrate processes, data stores, and external entities, helping in understanding system functionality at different levels of abstraction.

6.1.1 DFD Level 0 – Context Diagram

The Context Diagram (Level 0) represents the entire system as a single process and shows its interaction with external entities. It provides a high-level overview without going into internal details.

In the "BTech Food Delivery Wala" system, the main process is the Food Delivery System, which interacts with four external entities:

1. Customer

- Sends: Registration details, login credentials, food selections, cart data, delivery address, and payment requests
- Receives: Authentication status, food menu, cart details, order confirmation, and order status updates

2. Admin

- Sends: Food item data (add/remove/update), order status updates
- Receives: Customer orders, system reports, and food management data

3. Stripe Payment Gateway

- Sends: Payment confirmation or failure response
- Receives: Payment request details including amount and order information

4. MongoDB Database

- Stores and retrieves all system data
- Handles user data, food items, and order records

- The data flow between these entities ensures smooth communication and processing within the system.

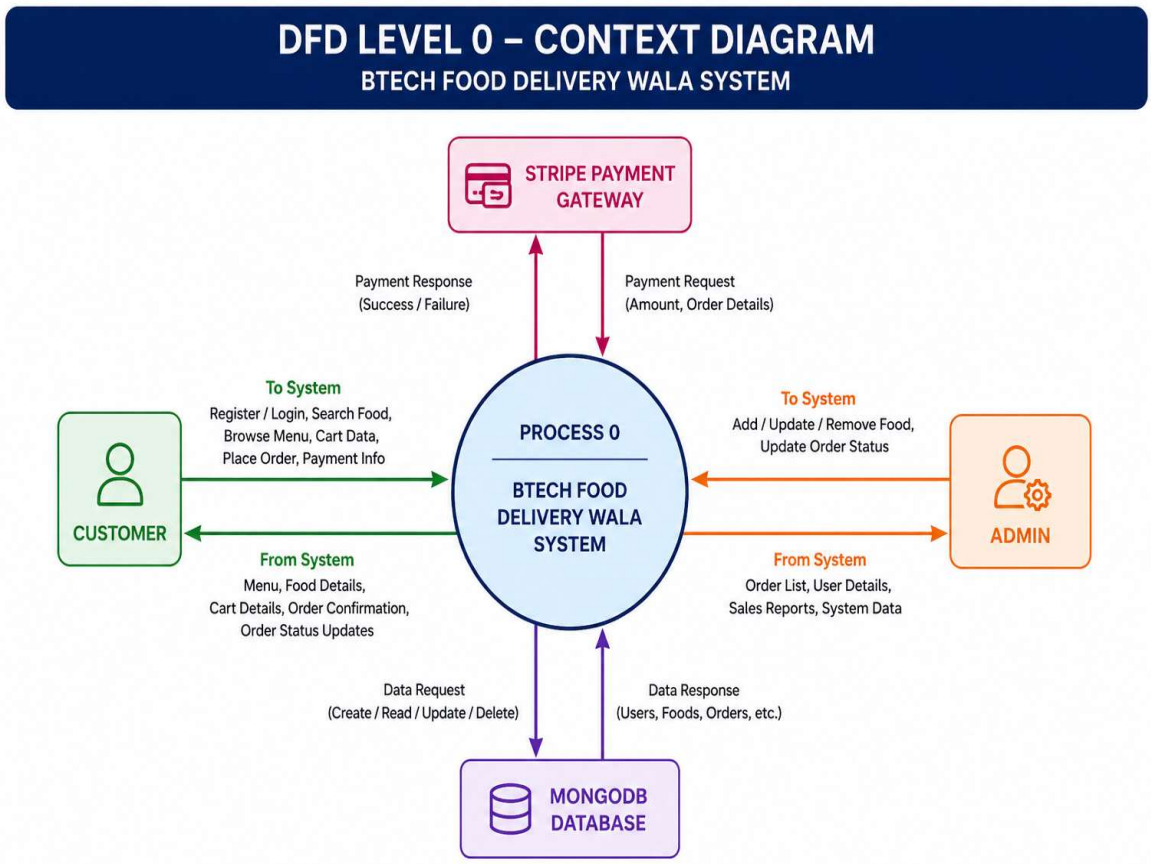


Fig 6.1: DFD Level 0 – Context Diagram

6.1.2 DFD Level 1 – Main System Processes

Level 1 DFD breaks down the system into major functional processes and shows how data flows between them and the data stores.

The system is divided into six main processes:

P1.0 User Authentication

- Handles user registration and login
- Verifies credentials using JWT and bcrypt
- Stores and retrieves user data from the Users data store

P2.0 Food Management

- Manages food items (add, update, delete)

- Retrieves food data for display
- Interacts with the Foods data store

P3.0 Cart Management

- Handles adding and removing items from the cart
- Maintains cart synchronization across sessions
- Updates cart data in the Users data store

P4.0 Order Processing

- Converts cart data into a confirmed order
- Handles order creation and storage
- Interacts with Orders data store

P5.0 Payment Verification

- Communicates with Stripe payment gateway
- Verifies payment status
- Updates order status based on payment outcome

P6.0 Admin Order Management

- Allows admin to view and manage all orders
- Updates order status in real time
- Retrieves and modifies data in Orders data store

Data Stores

The system uses three main data stores:

- D1: Users – Stores user details and cart data
- D2: Foods – Stores food item details
- D3: Orders – Stores order-related information

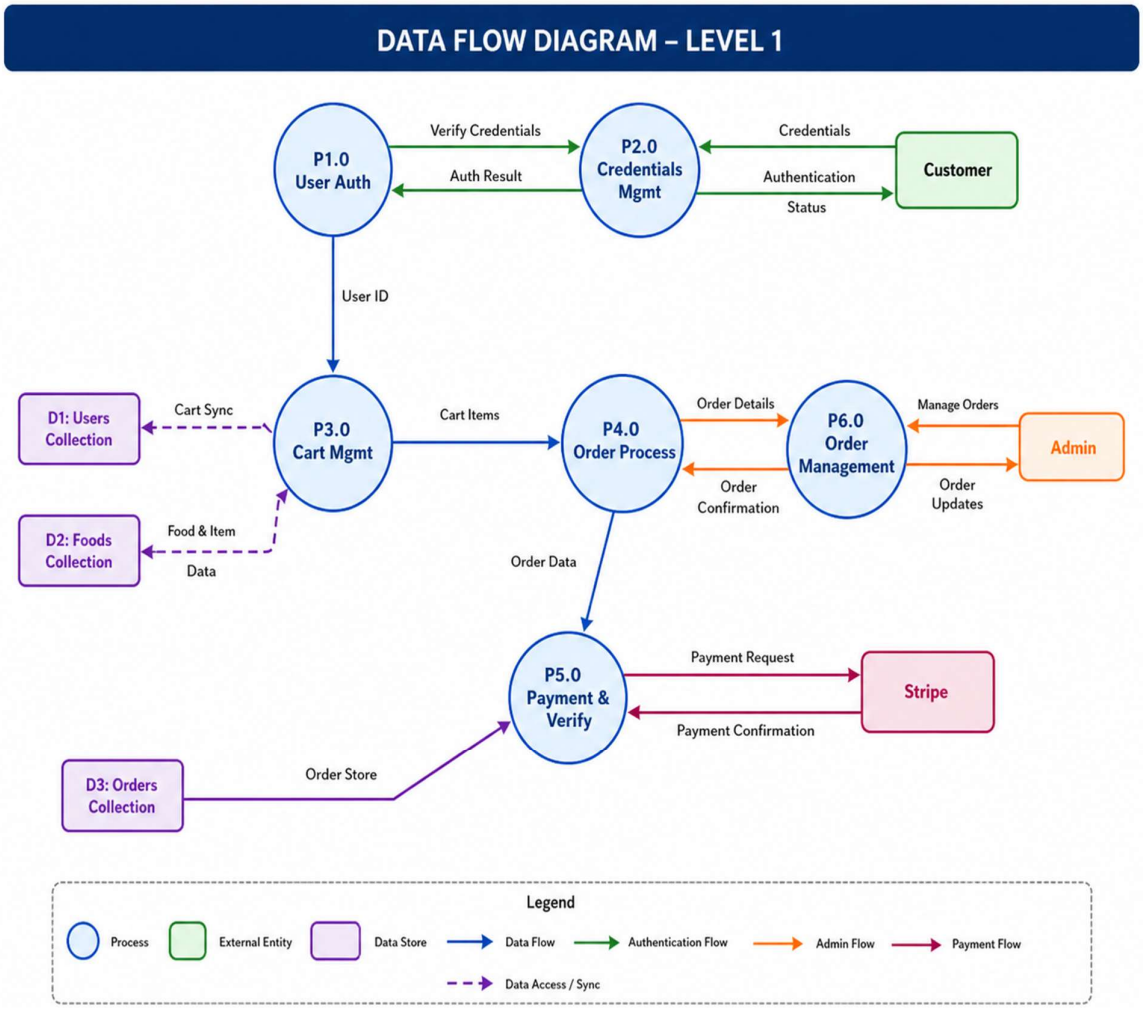


Fig 6.2: DFD Level 1 – Main System Processes

6.1.3 DFD Level 2 – Order Processing Expanded

- Level 2 DFD provides a detailed breakdown of a specific process. In this case, Process 4.0 (Order Processing) is expanded into smaller steps.

2.1 Validate Cart

- Checks whether the cart contains valid items
- Ensures quantities and prices are correct

2.2 Create Order in Database

- Generates a new order entry in MongoDB
- Stores user ID, items, total amount, and address

2.3 Clear User Cart

- Removes all items from the user's cart after order creation

- Updates cart data in the Users collection

2.4 Build Stripe Line Items

- Converts cart items into Stripe-compatible format
- Includes item name, quantity, and price

2.5 Create Stripe Session

- Initiates a payment session with Stripe
- Generates a secure payment URL

2.6 Verify Payment

- Confirms whether payment is successful
- Updates order status and payment flag accordingly
- This decomposition ensures clarity in how orders are processed step-by-step.

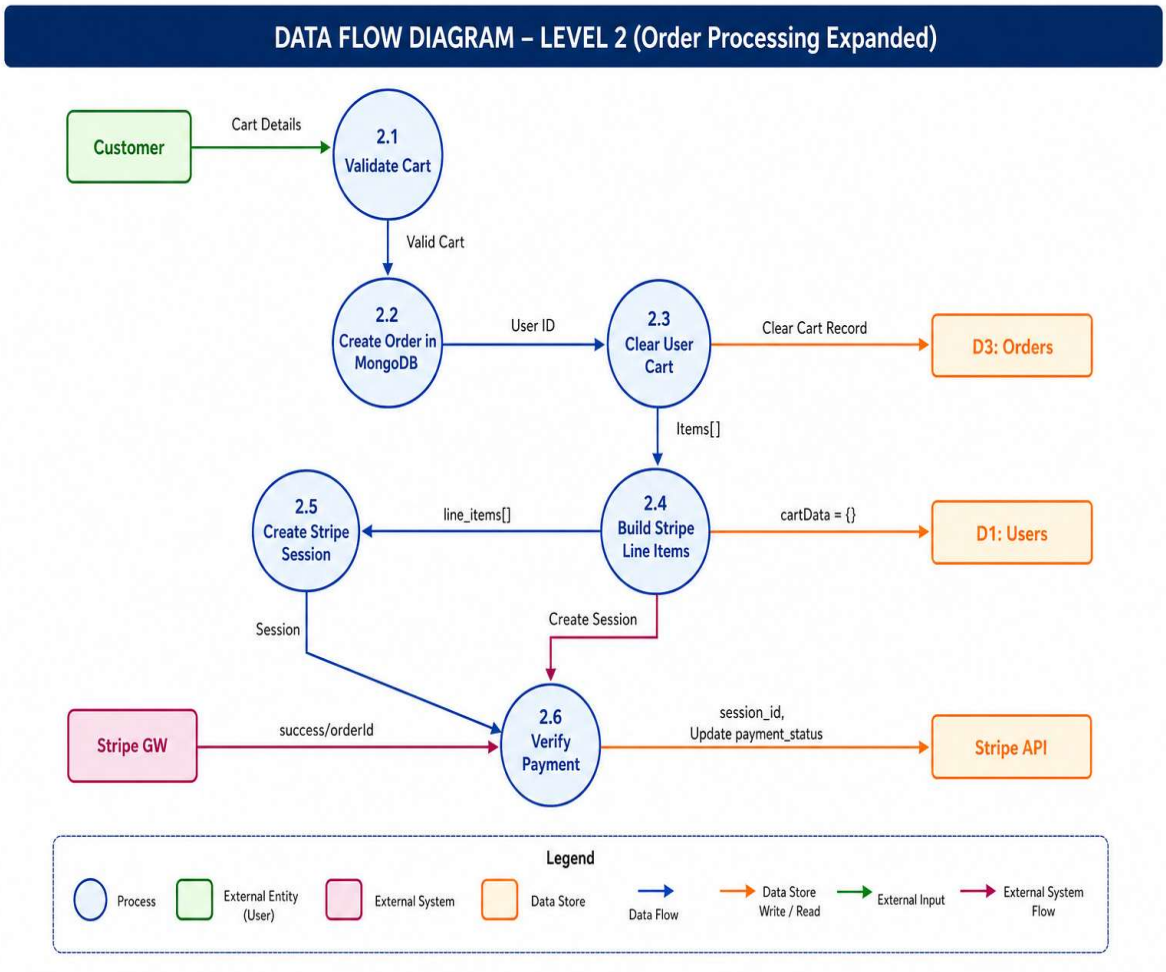


Fig 6.3: DFD Level 2 – Order Processing Detail

6.2 Entity Relationship (ER) Diagram

The ER Diagram represents the logical structure of the database by showing entities, attributes, and relationships.

Entities

1. USER

- Represents customers using the application
- Attributes: user ID, name, email, password, cart data

2. FOOD

- Represents food items available for ordering
- Attributes: food ID, name, description, price, category, image

3. ORDER

- Represents customer orders
- Attributes: order ID, user ID, items, total amount, address, status, date

4. ADMIN

- Represents system administrator
- Responsible for managing food items and orders

Relationships

- USER → ORDER (1:N)
  One user can place multiple orders, but each order belongs to one user
- ORDER → FOOD (M:N)
  One order can contain multiple food items, and one food item can appear in multiple orders
  (Implemented using an embedded array inside the Orders collection)
- ADMIN → FOOD (1:N)
  Admin can manage multiple food items
- ADMIN → ORDER (1:N)
  Admin can update and manage multiple orders

This structure ensures efficient data organization and retrieval.

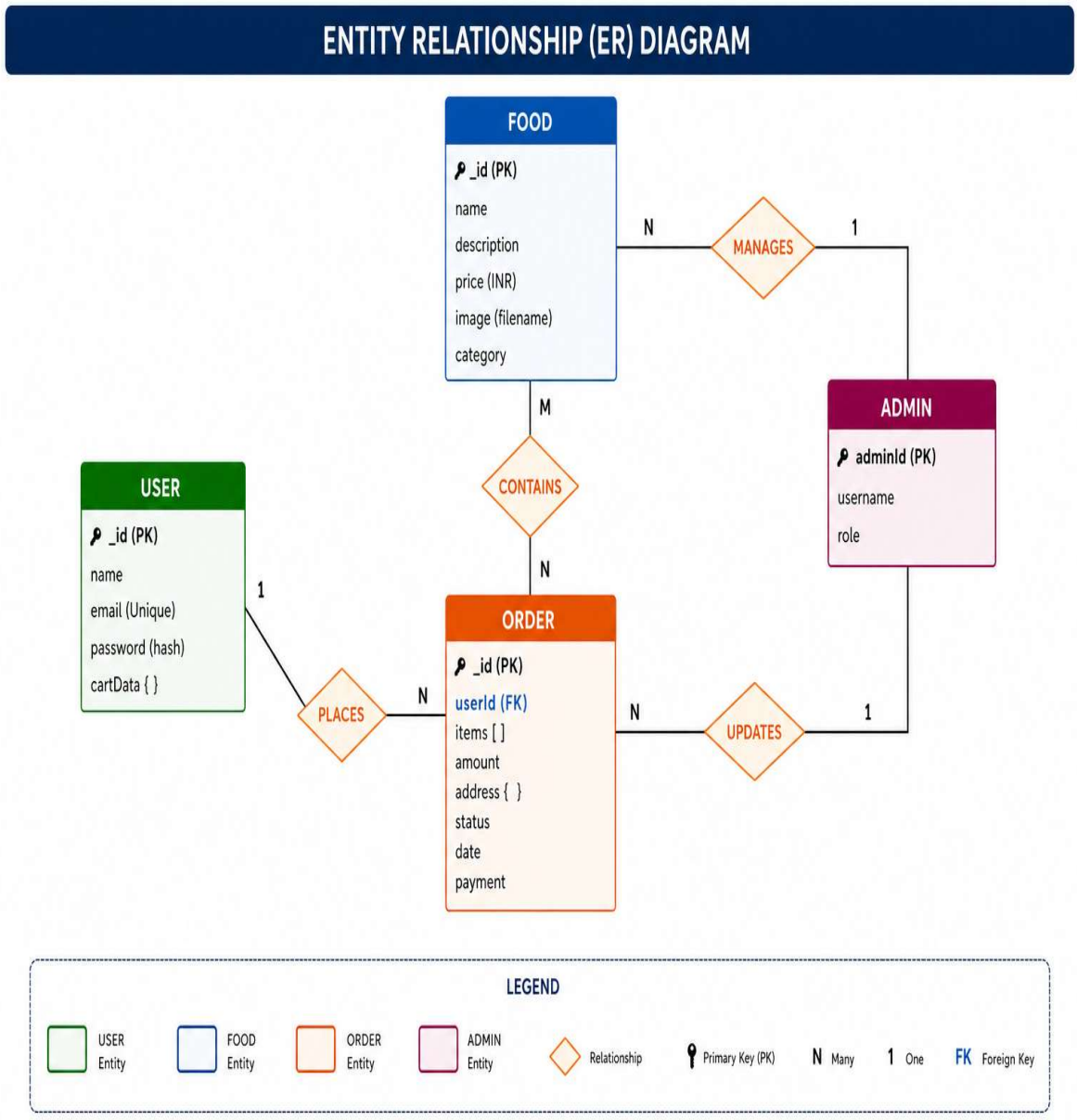


Fig 6.4: Entity Relationship (ER) Diagram

6.3 Data Structure – MongoDB Collections

The application uses MongoDB, a NoSQL database, which stores data in JSON-like documents. The schema is flexible and allows easy scalability.

6.3.1 Users Collection

Stores all registered user data.

Field	Type	Description
_id	ObjectId (PK)	Unique identifier
name	String	User's name
email	String	Unique email address
password	String	Hashed password using bcrypt
cartData	Object	Stores cart items (default: empty object)

Key Features:

- Email is unique to prevent duplicate accounts
- Password is securely hashed
- Cart data is embedded for faster access

6.3.2 Foods Collection

Stores all food items available in the system.

Field	Type	Description
_id	ObjectId (PK)	Unique identifier
name	String	Food item name
description	String	Food details
price	Number	Price in INR
image	String	Image filename/path

category	String	Food category

Key Features:

- Supports dynamic categories
- Stores image references for display
- Easily extendable for future attributes

6.3.3 Orders Collection

Stores all order-related data.

Field	Type	Description
_id	ObjectId (PK)	Unique identifier
userId	String (FK)	Reference to user
items	Array	List of ordered food items
amount	Number	Total order amount
address	Object	Delivery address
status	String	Order status
date	Date	Order creation time
payment	Boolean	Payment status

Default Values:

- status: “Food Processing”
- date: Current timestamp
- payment: false

Key Features:

- Supports multiple items per order
- Tracks payment and delivery status
- Enables order history tracking

Chapter 7: Detailed Design

Class Diagram, Sequence Diagram, I/O Description, Validation Checks, Program Code

7.1 Class Diagram

The Class Diagram represents the object-oriented structure of the system. It defines the classes, their attributes, methods, and relationships. The system consists of six major classes:

7.1.1. User Class

Represents application users (customers).

Attributes:

_id: ObjectId

name: String

email: String

password: String (hashed)

cartData: Object

Methods:

registerUser()

loginUser()

updateCart()

getUserData()

7.1.2. Food Class

Represents food items available in the system.

Attributes:

- _id: ObjectId
- name: String
- description: String
- price: Number
- category: String
- image: String

Methods:

- addFood()

- removeFood()
- listFoods()

7.1.3. Order Class

Represents customer orders.

Attributes:

- _id: ObjectId
- userId: String
- items: Array
- amount: Number
- address: Object
- status: String
- payment: Boolean

Methods:

- createOrder()
- updateStatus()
- getOrders()

7.1.4. StoreContext Class (Frontend - React)

Manages global state in the React application.

Attributes:

- cartItems: Object
- token: String
- foodList: Array

Methods:

- addToCart()
- removeFromCart()
- loadCartData()
- fetchFoodList()

7.1.5. AuthMiddleware Class

Handles authentication and authorization in the backend.

Attributes:

- token: String
- Methods:
- verifyToken()
- authorizeUser()

7.1.6. StripeService Class

Handles payment-related operations.

Attributes:

- stripeKey: String

Methods:

- createCheckoutSession()
- verifyPayment()

Class Relationships

User → Order (1:N): A user can place multiple orders

Order → Food (M:N): Each order contains multiple food items

StoreContext ↔ Food: Fetches and displays food data

StoreContext ↔ User: Manages authentication and cart

Backend classes interact with MongoDB for data persistence

StripeService integrates with Order for payment processing

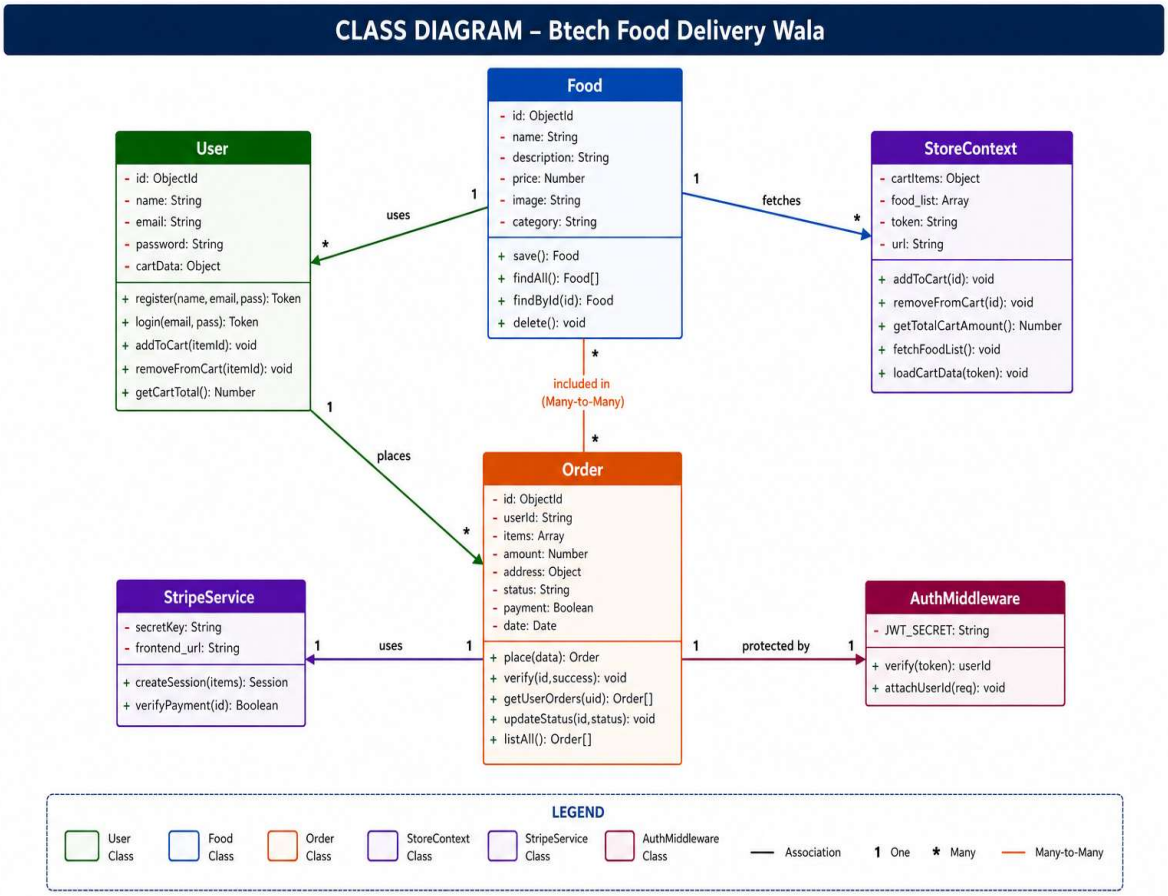


Fig 7.1: Class Diagram – Application Architecture

7.2 Sequence Diagram – Order Placement

The Sequence Diagram illustrates the step-by-step interaction between system components during order placement.

Actors Involved:

- Customer (Browser)
- React Frontend
- Express Backend
- MongoDB Database
- Stripe API

Flow of Events:

User Action

- Customer clicks “Place Order” from the cart page

Frontend Request

- React frontend sends order data (cart items + address) to backend API

Backend Processing

- Backend validates user authentication (JWT)
- Verifies cart items and calculates total amount

Database Interaction

- Order data is stored in MongoDB (Orders collection)

Stripe Integration

- Backend creates Stripe checkout session
- Sends payment details to Stripe API

Response to Frontend

- Stripe returns a session URL
- Frontend redirects user to Stripe checkout page

Payment Completion

- User completes payment on Stripe

Payment Verification

- Stripe sends confirmation response
- Backend verifies payment status

Order Update

- Order status updated to “Confirmed”
- Payment flag set to true

Final Response

- Frontend displays order confirmation
- User can view order in “My Orders”

Diagram Representation Notes:

- Solid arrows represent API calls or actions
- Dashed arrows represent responses
- Vertical lifelines show component activity duration

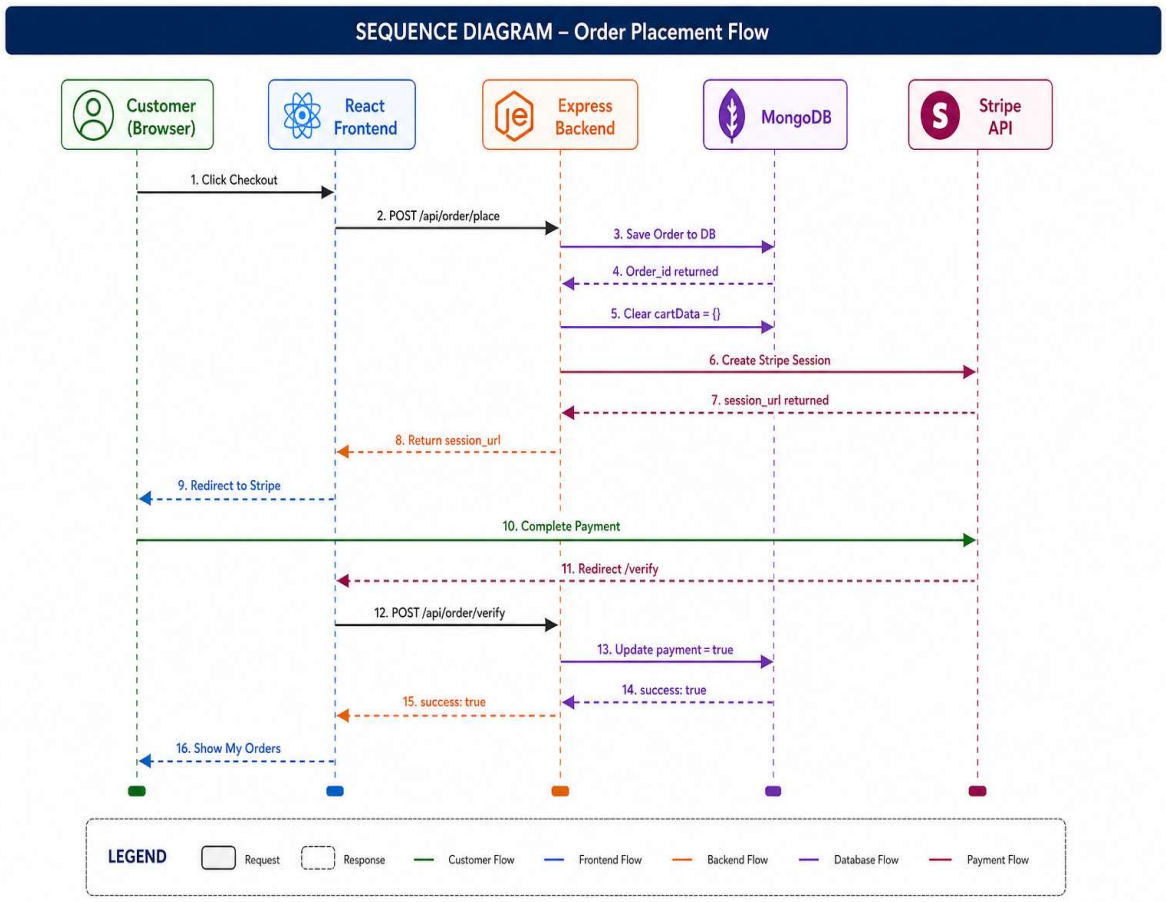


Fig 7.2: Sequence Diagram – Order Placement & Payment Flow

7.3 Input / Output Description

This section describes the inputs provided by users and the outputs generated by the system for each major interface of the "BTech Food Delivery Wala" application. It helps in understanding how users interact with the system and how the system responds to different actions.

Home Page:
The Home Page serves as the entry point of the application and provides an overview of available features and food items.
Output:

- Hero Banner: A visually appealing section displaying promotional content or featured offers
- Explore Menu Section: Category-based icons (e.g., Pizza, Burger, Drinks) allowing users to filter food items
- Food Display Grid:
  - Shows all available food items
  - Each item includes image, name, price, and short description
  - "Add to Cart" and "Remove" buttons for quick interaction
- Navbar:
  - Displays navigation links (Home, Cart, Orders, etc.)
  - Shows cart icon with real-time item count
  - Displays login/profile icon based on user status.

System Behavior:
- Fetches food data from backend API
- Dynamically updates UI using React state
- Reflects cart changes instantly

Login / Register Popup:
This module handles user authentication and account creation.
Input:
- Name: Required only during registration
- Email: Must be valid and unique
- Password: Must meet minimum security requirements

Output:
- On successful login/registration:
  - JWT (JSON Web Token) is generated
  - Token is stored in browser localStorage
  - Popup automatically closes
  - Navbar updates to show user profile icon
- On failure:
  - Error message displayed (invalid credentials or existing email)

System Behavior:
- Sends authentication request to backend
- Validates user credentials
- Maintains session using JWT

Cart Page:
The Cart Page displays all selected items before order placement.
Output:
- Item Table:
  - Food image
  - Item name
  - Price per item
  - Quantity selected
  - Total price per item
  - Remove option (delete item from cart)
- Cart Summary Section:
  - Subtotal (sum of all items)
  - Fixed delivery charge (₹150)
  - Grand total (subtotal + delivery)
- Action Button:
  - "Proceed to Checkout"

System Behavior:
- Retrieves cart data from global state/database
- Updates totals dynamically when items are added/removed
- Ensures accurate pricing before checkout

Place Order Page:
This page collects delivery details and initiates the payment process.
Input:
User is required to fill the following fields:
- First Name
- Last Name

- Email
- Street Address
- City
- State
- Zip Code
- Country
- Phone Number

Output:

- Displays order summary (items + total amount)
- On submission:
    - Redirects user to Stripe Checkout page
    - Initiates secure payment session

System Behavior:

- Validates all input fields
- Sends order data to backend
- Creates Stripe session for payment

My Orders Page:

This page allows users to view and track their past and current orders.

Output:

- Order Cards:
    - List of ordered items
    - Total amount
    - Number of items
    - Order date
    - Order status (e.g., Food Processing, Out for Delivery, Delivered)
- Status Indicator:
    - Visual representation (dot/color-based) of order status
- Track Order Button:
    - Refreshes order data from backend
    - Displays updated order status

System Behavior:

- Fetches order history from database
- Updates UI based on latest order status
- Enables real-time tracking (manual refresh)

Admin Panel:

The Admin Panel provides functionality for managing food items and customer orders.

A. Add Food Item

Input:

- Food image (uploaded using file input)
- Name
- Description
- Price
- Category

Output:

- Food item is added to the database
- Confirmation message displayed

System Behavior:

- Uses file upload middleware for image handling
- Stores data in Foods collection

B. Food List Management

Output:

- Table displaying all food items
- Columns include:
  - Image
  - Name
  - Price
  - Category
- Delete button for each item

System Behavior:

- Fetches food list from database
- Allows admin to remove items instantly

C. Order Management

Output:

- Displays all customer orders in a structured format
- Each order includes:
  - Customer details
  - Ordered items
  - Total amount
  - Current status
- Status Dropdown:
  - Options:
    - Food Processing
    - Out for Delivery
    - Delivered

System Behavior:

- Updates order status in real time
- Reflects changes immediately in customer view

7.4 Validation Checks

Validation ensures that user inputs and system operations are correct and secure.

Validation is a critical part of any web application as it ensures that the data entered by users is correct, complete, and secure before being processed or stored. Proper validation helps in preventing errors, improving user experience, and protecting the system from malicious activities.

- In the "BTech Food Delivery Wala" application, validation is implemented at multiple levels:
- Frontend Validation
- Backend Validation
- Security Validation
- This layered approach ensures data integrity and system reliability.

7.4.1 Frontend Validation

- Frontend validation is performed on the client side using React before sending data to the server. It provides immediate feedback to users and reduces unnecessary server requests.

Key Validations:

- Required Field Validation
  All mandatory input fields must be filled before form submission. This applies to login, registration, and order forms.
- Email Format Validation
  Ensures that the email entered follows a valid format (e.g., user@example.com).
- Password Strength Validation
  Checks that the password meets minimum length requirements (e.g., at least 6–8 characters).
- Numeric Validation
  Ensures that fields like phone number and zip code contain only numeric values.
- Real-Time Feedback
  Error messages are displayed instantly if invalid data is entered.
- Benefits:
- Improves user experience
- Reduces server load
- Prevents basic input errors early

7.4.2 Backend Validation

- Backend validation is performed on the server side using Node.js and Express.js. It acts as a second layer of validation to ensure that even if frontend validation is bypassed, invalid data cannot enter the system.

  Key Validations:

- JWT Token Verification
  Ensures that only authenticated users can access protected routes (e.g., placing orders, viewing cart).
- Duplicate Email Prevention
  Checks whether the email already exists in the database during registration.
- Data Type Validation
  Ensures that all incoming data matches expected formats (e.g., price as number, items as array).
- Order Amount Verification
  Recalculates the total amount on the server to prevent manipulation from the client side.
- Request Body Validation
  Verifies completeness of request data before processing.

  Benefits:

- Prevents unauthorized access
- Ensures data consistency

- Protects against client-side manipulation

7.4.3 Security Validation

- Security validation ensures that sensitive data is protected and the system is safeguarded against vulnerabilities and attacks.

Key Security Measures:

- Password Hashing (bcrypt)
  User passwords are encrypted before storing in the database, preventing exposure of plain-text passwords.
- Protected Routes (Middleware)
  Middleware functions verify authentication tokens before granting access to secure endpoints.
- Secure Payment Verification (Stripe)
  Payment confirmation is verified directly from Stripe to prevent fake or incomplete transactions.
- Environment Variables Protection
  Sensitive data such as API keys are stored securely using environment variables.

Field	Rule	Error Message	Layer
Name	Must not be empty	"Name is required"	Frontend
Email	Must follow valid format	"Invalid email address"	Frontend
Email	Must be unique	"Email already exists"	Backend
Password	Minimum 6–8 characters	"Password too short"	Frontend
Password	Must be hashed before storage	N/A (security process)	Security
Phone Number	Must contain only digits	"Invalid phone number"	Frontend
Zip Code	Must be numeric	"Invalid zip code"	Frontend
JWT Token	Must be valid and not expired	"Unauthorized access"	Backend
Cart Data	Must contain valid items	"Cart is empty or invalid"	Backend
Order Amount	Must match calculated total	"Invalid order amount"	Backend
Payment Status	Must be verified via Stripe	"Payment verification failed"	Security
Image Upload	Must be valid file type (jpg/png)	"Invalid image format"	Backend

7.5 Program Code

This section describes the core implementation of the "BTech Food Delivery Wala" application. The system follows a MERN stack architecture, where:

- Frontend is developed using React.js
- Backend is implemented using Node.js and Express.js
- Database is MongoDB
- Authentication is handled using JWT
- Payments are processed using Stripe

The following code snippets represent key components of the system.

7.5.1 Backend Entry Point – server.js

This file acts as the main entry point of the backend server. It initializes the Express application, connects to the database, and defines API routes.

```
import express from 'express'
import cors from 'cors'
import { connectDB } from './config/db.js'
import foodRouter from './routes/foodRoute.js'
import userRouter from './routes/userRoute.js'
import cartRouter from './routes/cartRoute.js'
import orderRouter from './routes/orderRoute.js'
import 'dotenv/config'

const app = express()

// Middleware
app.use(express.json())        // Parses incoming JSON requests
app.use(cors())                // Enables Cross-Origin Resource Sharing

// Database Connection
connectDB();

// API Routes
app.use('/api/food', foodRouter)
app.use('/images', express.static('uploads')) // Serves uploaded images
app.use('/api/user', userRouter)
app.use('/api/cart', cartRouter)
app.use('/api/order', orderRouter)

// Server Listener
app.listen(4000, ()=>console.log('Server on http://localhost:4000'))
```

Explanation:

- express.json() allows the server to accept JSON data from the frontend
- cors() enables communication between frontend and backend
- connectDB() establishes a connection with MongoDB Atlas
- Routes are modularized for better structure and maintainability
- Static folder (uploads) is used to serve food images

7.5.2 Auth Middleware – authMiddleware.js:

This middleware ensures that only authenticated users can access protected routes such as cart and order APIs.

```
import jwt from 'jsonwebtoken'

const authMiddleware = async (req, res, next) => {
    const { token } = req.headers;

    // Check if token exists
    if (!token)
        return res.json({success:false, message:'Not Authorized'})

    try {
        // Verify token using secret key
        const decoded = jwt.verify(token, process.env.JWT_SECRET);

        // Attach userId to request body
        req.body.userId = decoded.id;

        next(); // Proceed to next middleware/controller

    } catch(error) {
        res.json({success:false, message:'Error'})
    }
}
```

Explanation:

- Extracts JWT token from request headers
- Verifies token using secret key
- If valid, attaches userId to request
- Prevents unauthorized access to protected APIs

7.5.3 orderController.js – placeOrder:

```
const placeOrder = async (req, res) => {
  try {
    // Create new order
    const newOrder = new orderModel({
      userId:req.body.userId,
      items:req.body.items,
      amount:req.body.amount,
      address:req.body.address
    })

    // Save order in database
    await newOrder.save();

    // Clear user cart after placing order
    await userModel.findByIdAndUpdate(req.body.userId,{cartData:{}});

    // Prepare Stripe line items
    const line_items = req.body.items.map((item)=>({
      price_data:{
        currency:'inr',
        product_data:{name:item.name},
        unit_amount:item.price*100
      },
      quantity:item.quantity
    }))

    // Add delivery charges
    line_items.push({
      price_data:{
        currency:'inr',
        product_data:{name:'Delivery Charges'},
        unit_amount:15000
      },
      quantity:1
    })

    // Create Stripe checkout session
    const session = await stripe.checkout.sessions.create({
      line_items,
      mode:'payment',
      success_url:`${frontend_url}/verify?success=true&orderId=${newOrder._id}`,
      cancel_url:`${frontend_url}/verify?success=false&orderId=${newOrder._id}`,
    })

    // Send session URL to frontend
    res.json({ success:true, session_url:session.url })

  } catch(error){
    res.json({success:false,message:'Error'})
  }
}
```

Explanation:

- Creates a new order object and stores it in MongoDB
- Clears cart after order placement
- Converts cart items into Stripe-compatible format
- Adds delivery charges dynamically
- Creates Stripe checkout session
- Returns payment URL to frontend

7.5.4 StoreContext.jsx – Cart Functions:

```
const addToCart = async(itemId) => {
  setCartItems(prev=>({...prev,[itemId]:(prev[itemId]||0)+1}))
  if(token) await axios.post(url+'/api/cart/add',{itemId},{headers:{token}})
}

const getTotalCartAmount = () => {
  return Object.keys(cartItems).reduce((total,id)=>{
    const item = food_list.find(p=>p._id===id);
    return item ? total + item.price * cartItems[id] : total;
  },0);
}
```

Explanation:

- Updates cart state locally using React state
- Increments quantity if item already exists
- Syncs cart data with backend if user is logged in

7.5.5 Get Total Cart Amount Functions:

```
const getTotalCartAmount = () => {
       return Object.keys(cartItems).reduce((total,id)=>{
           const item = food_list.find(p=>p._id===id);
           return item ? total + item.price * cartItems[id] : total;
         },0);
       }
```

Explanation:

- Iterates through cart items
- Matches item IDs with food list
- Calculates total cost dynamically
- Used in cart and checkout pages

7.5.5 Overall Code Flow

- User interacts with frontend (React)
- API request sent using Axios
- Backend (Express) processes request
- Middleware validates authentication
- Controller performs business logic
- MongoDB stores/retrieves data
- Stripe handles payment
- Response sent back to frontend

7.5.5 Overall Code Flow

- User interacts with frontend (React)
- API request sent using Axios
- Backend (Express) processes request
- Middleware validates authentication
- Controller performs business logic

Chapter 8: Testing Techniques

Testing is a crucial phase in the software development lifecycle that ensures the system works correctly, efficiently, and securely. It helps in identifying bugs, validating functionality, and ensuring that the application meets user requirements.

For the "BTech Food Delivery Wala" application, testing was performed at multiple levels, including:

- API Unit Testing
- Integration Testing
- End-to-End Testing

The application was tested using tools such as **Postman** for backend API validation and modern web browsers (Chrome) for frontend and user flow testing.

8.1 Testing Strategy

The testing strategy adopted for this project focuses on verifying both **individual components** and the **complete system workflow**.

Approach Used:

1. **API Unit Testing (Backend Testing)**

- Each API endpoint was tested independently using Postman
- Ensured correct request handling and response format
- Validated authentication and database operations

2. **Frontend Integration Testing**

- Tested interaction between UI components and backend APIs
- Verified dynamic updates using React state and Context API

3. **End-to-End Testing**

- Tested complete user journey from login → cart → checkout → payment → order tracking
- Ensured seamless communication between frontend, backend, database, and Stripe

Scope of Testing:

- All 14 API endpoints were tested
- All major user flows were validated
- Both success and failure cases were considered

8.2 API Unit Testing

API Unit Testing focuses on validating backend endpoints individually. Each API was tested with different inputs to ensure correct behavior under various conditions.

Testing Tool Used:

- Postman

No	Endpoint	Method	Test Case	Expected	Result
1	/api/user/register	POST	Valid new user	Token returned	PASS
2	/api/user/register	POST	Duplicate email	Error message	PASS
3	/api/user/login	POST	Correct credentials	JWT token	PASS
4	/api/user/login	POST	Wrong password	success:false	PASS
5	/api/food/list	GET	Fetch all food	Food array	PASS
6	/api/food/add	POST	Add with image	Saved to DB	PASS
7	/api/food/remove	POST	Remove by ID	Deleted from DB	PASS
8	/api/cart/add	POST	Valid token	Cart updated	PASS
9	/api/cart/add	POST	No token	Not Authorized	PASS
10	/api/order/place	POST	Valid order	Stripe session	PASS
11	/api/order/verify	POST	success=true	payment=true	PASS
12	/api/order/status	POST	Update status	Status updated	PASS

Observations:

- JWT authentication worked correctly for protected routes
- Database operations (CRUD) were successfully executed
- Stripe session creation returned valid payment URLs
- Error handling worked as expected for invalid inputs

8.3 Integration Testing

Integration testing verifies how different modules work together, ensuring proper interaction between frontend, backend, and database.

Testing Focus:

- API integration with React frontend
- State synchronization (cart + orders)
- Admin panel interactions
- Real-time UI updates

No	Test Scenario	Expected Result	Result
1	User opens homepage without login	Menu displays correctly	PASS
2	Category filter click	Food list filters	PASS
3	Register via Login Popup	Token stored, profile shown	PASS
4	Add items to cart (logged in)	State + DB both updated	PASS
5	Checkout to Stripe	Stripe payment page loads	PASS
6	Payment success redirect	Order marked paid, My Orders	PASS
7	Admin adds food item	Item appears in customer portal	PASS
8	Admin updates order to Delivered	Customer sees updated status	PASS

Observations:

- Frontend and backend communication via Axios was successful
- Cart synchronization worked across sessions
- Admin updates were reflected instantly on the user side
- No major integration issues were found.

8.4 End-to-End Testing

End-to-End (E2E) testing ensures that the entire application works as expected from a user's perspective.

Complete User Flow Tested:

- User registers/login
- Browses food items
- Adds items to cart
- Proceeds to checkout
- Enters delivery details
- Completes payment via Stripe
- Redirected back to application
- Order appears in "My Orders"
- Admin updates order status
- User tracks updated order

Result:

- All flows executed successfully
- Payment integration worked without issues
- Order tracking reflected real-time updates

8.5 Error Handling Testing

Various error scenarios were tested to ensure system robustness.

Examples:

- Invalid login credentials → Proper error message
- Missing token → Unauthorized response
- Empty cart → Order blocked
- Payment failure → Order not marked as paid

8.6 Performance & Reliability Testing (Basic)

Although large-scale load testing was not conducted, basic checks were performed:

- API response time was fast under normal usage
- No UI lag observed during navigation
- Database queries executed efficiently

Chapter 9: Implementation & Maintenance

9.1 Implementation

The application is divided into three main parts:

- Backend (Server-side logic and APIs)
- Frontend (Customer interface)
- Admin Panel (Management interface)

9.1.1 Backend Implementation

The backend is developed using **Node.js and Express.js**, following a modular and RESTful architecture.

Key Features:

1. Express Server Setup

- The server is initialized using Express
- Middleware used:
    - express.json() → Parses incoming JSON requests
    - cors() → Enables cross-origin communication
- Server runs on port **4000**

2. Routing Structure

- APIs are divided based on resources:
    - /api/user → Authentication (register/login)
    - /api/food → Food management
    - /api/cart → Cart operations
    - /api/order → Order processing

This modular routing improves maintainability and scalability.

3. Database Integration

- MongoDB Atlas is used as a cloud database
- Mongoose ODM is used to define schemas:
    - Users
    - Foods
    - Orders

- Data is stored in JSON-like BSON format

4. Authentication System

- JWT (JSON Web Token) is used for stateless authentication
- Tokens are verified using middleware
- User ID is extracted and used in protected routes

5. File Upload Handling

- Multer is used for handling multipart/form-data
- Food images are stored in the uploads directory
- Images are served via static route /images

6. Payment Integration

- Stripe API is integrated for online payments
- Checkout session is created dynamically in placeOrder
- Payment verification ensures secure transactions

7. Environment Configuration

- Sensitive data (API keys, JWT secret, DB URI) stored using dotenv
- Improves security and environment flexibility

Backend Flow Summary:

1. Client sends request via Axios
2. Express route receives request
3. Middleware validates authentication
4. Controller executes logic
5. Database operation performed
6. Response sent back to client

9.1.2 Frontend Implementation

The frontend is developed using **React.js with Vite** for fast development and optimized builds.

1. Application Structure

- Component-based architecture
- Pages include:
    - Home
    - Cart

- Place Order
- My Orders

2. Global State Management

- Implemented using **React Context API (StoreContext)**
- Manages:
  - Cart items
  - Authentication token
  - Food list

3. Routing

- Implemented using React Router
- All routes are defined in App.jsx
- Enables Single Page Application (SPA) behavior

4. API Communication

- Axios is used to communicate with backend
- Base URL: http://localhost:4000
- Token passed in headers for protected routes

5. Cart Functionality

- Cart state maintained globally
- Real-time updates using React state
- Cart indicator (dot) logic:
  - Displayed when getTotalCartAmount() > 0

6. UI/UX Features

- Responsive design using CSS
- Dynamic rendering of food items
- Interactive add/remove cart buttons

9.1.3 Admin Panel Implementation

The Admin Panel is a separate application built using **React + Vite**, running on a different port.

Key Features:

- Runs on **port 5174** during development
- Independent UI for administrative operations

1. Food Management

- Add food items using form
- Upload images using **FormData**
- Delete existing items

2. Order Management

- View all customer orders
- Update order status via dropdown:
    - Food Processing
    - Out for Delivery
    - Delivered

3. Notifications

- Uses **React Toastify** for:
    - Success messages
    - Error alerts

4. Backend Integration

- Uses Axios for API calls
- Interacts with same backend as customer app

9.2 Deployment Architecture

The application is designed for both development and production environments.

Component	Development	Production	Port
Customer Frontend	Vite Dev Server	Vercel	5173
Admin Panel	Vite Dev Server	Vercel	5174
Backend API	Node.js localhost	Render / Railway	4000
Database	MongoDB Atlas	MongoDB Atlas	Cloud
Payment	Stripe Test Mode	Stripe Live Mode	External

Architecture Overview:

- User interacts with frontend hosted on Vercel
- Frontend sends API requests to backend (Render/Railway)
- Backend communicates with MongoDB Atlas

- Stripe handles payment processing externally
- Response flows back to frontend

Advantages of Deployment:

- Scalable cloud-based architecture
- Separation of frontend and backend
- High availability and reliability
- Easy CI/CD deployment

9.3 Maintenance Plan

Regular: MongoDB Atlas automated backups, npm audit for dependency security updates. Security: JWT secret rotation, HTTPS enforcement, CORS whitelist updates. Performance: MongoDB indexes on userId (orders) and email (users).

9.3.1 Regular Maintenance

- **Database Backups:**
  - Automated backups using MongoDB Atlas
  - Prevents data loss
- **Dependency Updates:**
  - Regular checks using npm audit
  - Fix vulnerabilities in packages

9.3.2 Security Maintenance

- **JWT Secret Rotation:**
  - Periodically update secret keys
  - Prevent token misuse
- **HTTPS Enforcement:**
  - Secure communication between client and server
- **CORS Policy Updates:**
  - Restrict access to trusted domains only
- **Environment Variable Protection:**
  - Keep sensitive data secure

9.3.3 Performance Optimization

- **Database Indexing:**

  - Index on email (Users collection)
  - Index on userId (Orders collection)
- **API Optimization:**
  - Reduce unnecessary database queries
  - Optimize response times
- **Caching (Future Scope):**
  - Use caching for frequently accessed data

9.3.4 Bug Fixing & Updates

- Continuous monitoring for bugs
- User feedback integration
- Regular feature enhancements

Chapter 10: Limitations, Future Scope & Enhancements

10.1 Limitations

Despite successfully implementing core features, the current system has the following limitations:

1. Localhost-Based Deployment

- The application currently runs on a local development server
- Not accessible over the internet without deployment
- Limits real-world usability and scalability

2. No Admin Authentication

- Admin panel is not secured with login authentication
- Any user with access can perform admin operations
- Creates potential security vulnerabilities

3. Promo Code Functionality (UI Only)

- Promo code input field is present in UI
- No backend logic implemented for validation or discount calculation
- Does not affect actual order pricing

4. Single Restaurant Model

- The system supports only one restaurant
- No option to onboard multiple vendors
- Limits scalability and business expansion

5. No Real-Time GPS Tracking

- Users cannot track delivery location in real time
- Order tracking is limited to status updates only
- Reduces user engagement and transparency

6. No Customer Reviews & Ratings

- Users cannot provide feedback on food items
- No rating system for quality or service
- Limits user interaction and decision-making support

7. No Push Notifications

- Users are not notified about:
    - Order confirmation
    - Order status updates
- Requires manual checking of order status

8. Local Image Storage

- Food images are stored on the server filesystem (uploads folder)
- Not scalable for production environments
- Risk of data loss if server crashes

10.2 Future Scope & Enhancements

To transform this project into a production-level application, several enhancements can be implemented.

1. Mobile Application Development

- Develop mobile apps using **React Native**
- Support both Android and iOS platforms
- Improve accessibility and user engagement

2. Multi-Restaurant Support

- Allow multiple restaurants to register and manage their menus
- Separate admin panels for each vendor
- Implement restaurant-based filtering for users

3. Real-Time Order Tracking

- Use **Socket.io (WebSockets)** for real-time communication
- Integrate GPS APIs for live location tracking
- Display delivery agent movement on map

4. Customer Reviews & Ratings System

- Allow users to rate food items and services
- Display average ratings and reviews
- Improve decision-making for customers

5. Push Notifications Integration

- Use **Firebase Cloud Messaging (FCM)**
- Send real-time notifications for:
    - Order confirmation

- Dispatch updates
- Delivery completion

6. Cloud-Based Image Storage

- Replace local storage with:
  - Cloudinary or
  - AWS S3
- Benefits:
  - Better scalability
  - Faster image delivery
  - Improved reliability

7. Admin Authentication with Role-Based Access Control (RBAC)

- Secure admin panel with login system
- Implement roles such as:
  - Super Admin
  - Restaurant Admin
- Restrict access based on permissions

8. Promo Code Backend Implementation

- Add backend logic for:
  - Discount validation
  - Expiry dates
  - Usage limits
- Apply discounts dynamically during checkout

9. AI-Based Food Recommendation System

- Use machine learning techniques such as:
  - Collaborative filtering
  - User behavior analysis
- Suggest personalized food items to users

10. Admin Analytics Dashboard

- Provide insights such as:
  - Total sales
  - Popular food items
  - Customer activity

Chapter 11: References & Bibliography

11.1 References

No	Title	Author/Source	URL/Year
1	React.js Official Documentation	Meta/React Team	https://react.dev
2	Node.js Official Documentation	OpenJS Foundation	https://nodejs.org
3	Express.js Guide	Express.js Team	https://expressjs.com
4	MongoDB Official Documentation	MongoDB Inc.	https://mongodb.com/docs
5	Mongoose ODM Documentation	Automattic	https://mongoosejs.com
6	Stripe Payment API Docs	Stripe Inc.	https://stripe.com/docs
7	JSON Web Tokens	Auth0	https://jwt.io
8	React Router DOM v6	Remix Software	https://reactrouter.com
9	OWASP Web Security Guide	OWASP Foundation	https://owasp.org
10	Axios HTTP Client Docs	Axios Project	https://axios-http.com

11.2 Bibliography

1. Traversy, B. (2023). MERN Stack Front to Back. Udemy.

2. Chodorow, K. (2019). MongoDB: The Definitive Guide. O'Reilly Media.

3. Banks, A. & Porcello, E. (2020). Learning React. O'Reilly Media.

4. Banerjee, S. & Gupta, A. (2020). Online Food Delivery Analysis. IJMS, 7(2), 45-58.

5. AKTU. B.Tech Final Year Project Guidelines, CSE. Lucknow, UP (2025).

Plagiarism Report

PLAGIARISM DECLARATION

I hereby declare that the project titled **"Online Food Delivery System"** is an original work carried out by us as part of our Bachelor of Technology (B.Tech) curriculum. This project has been developed using the MERN stack (MongoDB, Express.js, React.js, Node.js) and has not been copied from any unauthorized sources.

All the content, including code, design, and documentation presented in this report, is our own work. Wherever references have been taken from books, research papers, or online resources, they have been properly cited and acknowledged.

We further declare that this project has not been submitted to any other university or institution for the award of any degree or diploma.

Name: Bharat Kumar, Vishvjeet Gupta, Ashish Kumar, Yadvesh Yadav

Course: Bachelor of Technology (B.Tech)

Branch: Computer Science and Engineering (CSE) – 8th Semester

Institution: SR Institute of Management & Technology, Lucknow, UP

Date: _____

Signatures:

Bharat Kumar — _____

Vishvjeet Gupta — _____

Ashish Kumar — _____

Yadvesh Yadav — _____

Research Paper Details

RESEARCH PAPER

Online Food Delivery System Using MERN Stack

Bharat Kumar, Vishvjeet Gupta, Ashish Kumar, Yadvesh Yadav
B.Tech (CSE) – 8th Semester, SR Institute of Management & Technology, Lucknow, UP
Affiliated to Dr. A.P.J. Abdul Kalam Technical University, Lucknow, UP – 2025-2026

Abstract

The rapid growth of digital technology has transformed the way people order food. This research paper presents the design and development of an Online Food Delivery System using the MERN stack (MongoDB, Express.js, React.js, Node.js). The system provides a user-friendly interface for customers to browse food items, place orders, and manage their accounts efficiently. The application aims to simplify the food ordering process while ensuring scalability, performance, and security.

1. Introduction

With the increasing use of smartphones and web applications, online food delivery systems have become highly popular. Traditional methods of ordering food via phone calls are being replaced by modern web-based platforms. This project focuses on developing a responsive and efficient food delivery application that enhances user experience and reduces manual effort. The Btech Food Delivery Wala application demonstrates how the MERN stack can be effectively used to build a complete, production-grade food ordering system.

2. Literature Review

Various online food delivery platforms such as Swiggy and Zomato have revolutionized the industry. Existing systems provide features like real-time tracking, online payment, and restaurant listings. However, smaller systems often lack customization and scalability. This project aims to build a simplified yet effective system using modern technologies. The MERN stack has been identified in academic literature as a highly suitable combination for building scalable single-page web applications with real-time data requirements (Traversy, 2023; Kumar, 2021).

3. Methodology

The application is developed using the MERN stack with the following technology assignments:

- **MongoDB:** NoSQL cloud database (Atlas) for storing users, food items, and orders
- **Express.js:** RESTful API backend framework handling all business logic
- **React.js:** Component-based frontend UI library with Context API for state management

- **Node.js:** Non-blocking I/O JavaScript runtime for server-side operations

The development follows a modular approach with separate frontend (port 5173), admin panel (port 5174), and backend services (port 4000) communicating via REST APIs. JWT tokens handle authentication and Stripe processes payments securely.

4. System Design

The system consists of three integrated components:

- **User Module:** Registration, login with JWT, browse food by category, add/remove from cart, place order via Stripe, track orders in My Orders page
- **Admin Module:** Add food items with image upload (Multer), remove food items, view all customer orders, update order delivery status
- **Database:** MongoDB Atlas with three collections — Users (with embedded cartData), Foods, Orders — managed through Mongoose ODM

The architecture follows a three-tier client-server model (Presentation Layer → Application Layer → Data Layer) ensuring smooth, decoupled communication between frontend and backend via Axios HTTP requests.

5. Features of the Application

- User authentication (Login/Register) with JWT and bcrypt password hashing
- Browse food items with category-based filtering (8 categories)
- Add to cart, remove from cart, persistent cart stored in MongoDB
- Order placement with delivery address form and Stripe payment gateway (INR)
- Admin panel: add/remove food items with image upload, manage all orders
- Order status tracking: Food Processing → Out for Delivery → Delivered
- Responsive UI with smooth scroll and intuitive navigation
- Secure backend APIs with Auth Middleware (JWT verification on protected routes)

6. Results and Discussion

The developed system successfully performs all food delivery operations. It provides a smooth user experience with fast page loading (1-2 seconds via Vite bundling) and fully responsive design. The MERN stack ensures scalable data handling with MongoDB Atlas supporting concurrent users. All 14 API endpoints were tested and passed with Postman. Integration testing confirmed seamless communication between frontend, backend, and Stripe. The admin panel provides real-time order management with status updates visible to customers instantly.

7. Conclusion

The Online Food Delivery System simplifies the process of ordering food through a digital platform. It reduces manual work and improves efficiency for both customers and restaurant administrators. The use of the MERN stack ensures better performance, scalability, and maintainability. The application successfully demonstrates how modern web technologies can solve real-world problems, and the project team has gained hands-on experience in full-stack development, REST API design, database modeling, payment gateway integration, and software project management.

8. Future Scope

- Integration of additional online payment gateways (Razorpay, PayPal)
- Real-time GPS-based order tracking using Socket.io and Google Maps API
- Mobile application development using React Native for Android and iOS
- AI-based food recommendations using collaborative filtering on order history
- Multi-restaurant support with separate admin panels per restaurant

9. References

1. MERN Stack Documentation – https://www.mongodb.com/mern-stack
2. MongoDB Official Website – https://www.mongodb.com/docs
3. React.js Official Documentation – https://react.dev
4. Node.js Official Documentation – https://nodejs.org/en/docs
5. Stripe Payment Gateway Documentation – https://stripe.com/docs
6. Express.js Framework Documentation – https://expressjs.com
7. Traversy, B. (2023). MERN Stack Front to Back. Udemy Online Course.
8. Banerjee, S. & Gupta, A. (2020). Analysis of Online Food Delivery Platforms in India. IJMS, 7(2), 45–58.

Appendix in Report

Appendix A – API Reference

Endpoint	Method	Auth	Description
/api/user/register	POST	None	Register new user
/api/user/login	POST	None	Login, returns JWT token
/api/food/list	GET	None	Get all food items
/api/food/add	POST	Admin	Add food with image upload
/api/food/remove	POST	Admin	Delete food item and image
/api/cart/add	POST	JWT	Add item to user cart in MongoDB
/api/cart/remove	POST	JWT	Decrement cart item quantity
/api/cart/get	POST	JWT	Get user cart from MongoDB
/api/order/place	POST	JWT	Place order + Stripe Checkout Session
/api/order/verify	POST	None	Verify Stripe payment result
/api/order/userorders	POST	JWT	Get orders for logged-in user
/api/order/list	GET	Admin	Get all orders for admin
/api/order/status	POST	Admin	Update order delivery status

Appendix B – Project Team

No	Name	Role	Dept	Year
1	Bharat Kumar	Team Lead / Full Stack	CSE	Final Yr
2	Vishvjeet Gupta	Backend Developer	CSE	Final Yr
3	Ashish Kumar	Frontend Developer	CSE	Final Yr
4	Yadvesh Yadav	DB & Admin Panel	CSE	Final Yr

--- End of Project Report ---

Food Delivery Wala | SR Institute of Management & Technology | CSE | 2025-2026